

1 P1 — Lean formalization appendix

This appendix prints the complete published Lean sources for P1 in the exact dependency order used by the package verifier. The source snapshot is Git commit [29fbb70da18564048f62492e43870f2f96a65463](#), which matched the local `origin/main` tracking ref when this appendix was prepared. Every file below was read from that commit and checked byte for byte against the working copy and the recorded [source hashes](#). Links to individual files are pinned to the same commit.

1.1 Scope and proof boundary

The [P1 package overview](#) and [final written proof](#) mark C1–C5 **Solved by complete written arguments** under the task’s written-proof hand-in rule. This appendix reports supporting Lean formalizations, **not** full Lean proofs of those five general statements. The package does not contain assembled general C1–C5 Lean theorems. C6 was skipped. No hackathon submission or organizer acceptance is claimed.

Cell	Complete written result	Scope certified by these Lean modules
C1	All N planar lines, including empty and repeated configurations	$N = 2$ via <code>angle2_le</code> , and named $N = 3, 4$ theorems in <code>Angles.lean</code>
C2	All $m \geq 2$ with non-neighbors orthogonal	$m = 2$ in $\mathbb{R}^1$ and $m = 3$ in $\mathbb{R}^2$ in <code>Angles.lean</code>
C3	All $d \geq 1$	$d = 1, 2$ in <code>Angles.lean</code>
C4	Five lines in $\mathbb{R}^3$ and six in $\mathbb{R}^4$, both sharp	Analytic and coordinate lemmas, compact maximizer, sparse degree-two reduction, corank bounds, and cycle inequalities; the final whole-cell inequalities are written proofs
C5	All $d \geq 2$ with $N = d + 2$, sharp	$d = 2$ and shared C4 lemmas; the general induction is a written proof
C6	Skipped	None

The [criteria audit](#) maps the complete written arguments to the live cell statements. The source files use neither `sorry`, `admit`, custom `axiom`, `unsafe`, nor `native_decide` to assert completion. The [fresh local replay record](#) reports successful sequential compilation of these exact 18 files and 51 `#print` axioms declarations; their reported axioms were only `propext`, `Classical.choice`, and `Quot.sound`. That replay reused the pinned Mathlib checkout; it was not a clean network download.

1.2 Reproduction and exact source hashes

The package pins Lean **4.34.1** in [lean-toolchain](#) and Mathlib commit `d13f23b723b8a846827a245b89c10fc7d3f11612` in [lakefile.toml](#). From `cagent/p1` in the published repository, the portable replay command is:

```
./verify.sh
```

The [verification script](#) fetches required Mathlib dependencies and compiles the following modules in order. The hashes are SHA-256 of each complete .lean file as read from the pinned Git commit:

```
Angles.lean a4f6c5b156b5b6630cb3c85bc0b342e3b32eb9957368a6cd7e79871c4bb019fe
C4Pentagon.lean 092e302e13dcc62784280594511a29a058f6b8191c56fac98ae09a18e33f964d
C4Extremum.lean 418cf7266191c243478469276cd84d95f054942949e478b8bc78d9eed9a33ba9
C4Rotation.lean af61ab1e5d9de2d496184a668e5bf66e8687b60d869f17ecf4dddf64685f8ae3a
C4SparseStep.lean 072592200ed4d8059b3a1538911a0e2c7dba50ec4ff738ea105db497cee02272
C4SignedSparse.lean 5bdcda7a554ed29ed6221743fe95a077ba52aba441239e0f6024bdc526db84a2
C4Geometry.lean d08fae0bc5dd30adc77f62237269fd36f9e0c1fb2c3c56d0a0d48c5cd014dc4f
C4LocalRotation.lean e87a6df1b1713b3192a22afcedbc00bf210a0be7f51b50a4c3e2f655060635a0
C4Selection.lean 768979c102b411b643bdf02eb47f8ef0b7535aeab151f745baaa00ae556e6720
C4Configuration.lean f43198c702add47aeb0477c1b85979a8b576e68b6c4cfd3035d6e155de3991d5
C4Replacement.lean 4d9286153ac302d4bf6d93e87418bffa41790380e034b574cc7667cd70222d3
C4Direction.lean a1b1b936ccf804d516739f861176d80e25368529f39af3cdd8d18c564005c953
C4MaximalSparse.lean 370d792a5e0d7b6b1b56dfbdf57577b68eac91b9690b8b328caae6019513218
C4NeighborCount.lean 9d4f9fa2f09f805192d2ac8b9d9a3e9801615b11b0f513fa83e0c1abb5d74833
C4SparseConfiguration.lean 7bd4a7aa9e3b8b48313688d6e6126ab219cf60c54ad275089fe8e9fd050f9932
C4MatrixCorank.lean 180d00723bd73798394275b59b01dfa85a2c797b335e8267dd121993b5224cee
C4FiveCycle.lean fe0bb40d9fc9b71f42d69ef814c7199db5a214bfbcee507addca3340f8d4aed7
C4SixCycle.lean 2dc09cb3ac16e054a07e888b0e9aa527f541f4537c84b906b34ec0646df1be6f
```

1.3 Complete Lean source, in dependency order

1.3.1 1. Angles.lean

[Pinned source file](#) · SHA-256 a4f6c5b156b5b6630cb3c85bc0b342e3b32eb9957368a6cd7e79871c4bb019fe

```
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Inverse
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Ring
import Mathlib.Tactic.NormNum

/--! Genuine Euclidean cases of P1. Vectors use Cartesian coordinates;
`unit2` is exactly squared Euclidean norm = 1. No general cell claimed here. -/
namespace Angles

abbrev Vec2 := ℝ × ℝ

def dot2 (x y : Vec2) : ℝ := x.1 * y.1 + x.2 * y.2
def unit2 (x : Vec2) : Prop := dot2 x x = 1
noncomputable def angle2 (x y : Vec2) : ℝ := Real.arccos |dot2 x y|

theorem angle2_nonneg (x y : Vec2) : 0 ≤ angle2 x y := Real.arccos_nonneg _
theorem angle2_le (x y : Vec2) : angle2 x y ≤ Real.pi / 2 :=
  Real.arccos_le_pi_div_two.mpr (abs_nonneg _)

theorem dot2_bound {x y : Vec2} (hx : unit2 x) (hy : unit2 y) :
  |dot2 x y| ≤ 1 := by
  dsimp [unit2, dot2] at *
  have h := sq_nonneg (x.1 * y.2 - x.2 * y.1)
  have hid : (x.1*y.1+x.2*y.2)^2 + (x.1*y.2-x.2*y.1)^2 =
    (x.1*x.1+x.2*x.2)*(y.1*y.1+y.2*y.2) := by ring
  rw [hx, hy] at hid
  rw [abs_le]
```

```

constructor <=> nlinarith [sq_nonneg (x.1*y.1+x.2*y.2+1),
  sq_nonneg (x.1*y.1+x.2*y.2-1)]

theorem gram2 (x y z : Vec2) :
  dot2 x x * dot2 y y * dot2 z z + 2 * dot2 x y * dot2 y z * dot2 x z -
  dot2 x x * (dot2 y z)^2 - dot2 y y * (dot2 x z)^2 -
  dot2 z z * (dot2 x y)^2 = 0 := by
dsimp [dot2]
ring

theorem arccos_pair {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b)
  (h : a^2 + b^2 = 1) : Real.arccos a + Real.arccos b = Real.pi / 2 := by
have hs : Real.sqrt (1 - a^2) = b := by
  rw [show 1-a^2 = b^2 by nlinarith, Real.sqrt_sq hb]
rw [Real.arccos_eq_arcsin ha, hs, Real.arccos_eq_pi_div_two_sub_arcsin]
ring

/-- C2, full geometric case m=3 (not arbitrary m). -/
theorem c2_m3 {x y z : Vec2} (hx : unit2 x) (hy : unit2 y) (hz : unit2 z)
  (hxyz : dot2 x z = 0) : angle2 x y + angle2 y z = Real.pi / 2 := by
have hg := gram2 x y z
change dot2 x x = 1 at hx
change dot2 y y = 1 at hy
change dot2 z z = 1 at hz
rw [hx, hy, hz, hxyz] at hg
apply arccos_pair (abs_nonneg _) (abs_nonneg _)
simp only [sq_abs] using (show (dot2 x y)^2 + (dot2 y z)^2 = 1 by nlinarith [hg])

/-- Analytic certificate used for three arbitrary planar lines. -/
theorem arccos_triple {a b c : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) (hc : 0 ≤ c)
  (ha1 : a ≤ 1) (hb1 : b ≤ 1) (_hc1 : c ≤ 1)
  (h : 1 ≤ a^2 + b^2 + c^2 + 2*a*b*c) :
  Real.arccos a + Real.arccos b + Real.arccos c ≤ Real.pi := by
have hsa : (Real.sqrt (1-a^2))^2 = 1-a^2 :=
  Real.sq_sqrt (by nlinarith)
have hsb : (Real.sqrt (1-b^2))^2 = 1-b^2 :=
  Real.sq_sqrt (by nlinarith)
have hsab : Real.sqrt (1-a^2) * Real.sqrt (1-b^2) ≤ c+a*b := by
  have hprod : (Real.sqrt (1-a^2) * Real.sqrt (1-b^2))^2 =
    (1-a^2)*(1-b^2) := by rw [mul_pow, hsa, hsb]
  have hnn := mul_nonneg ha hb
  have hsn := mul_nonneg (Real.sqrt_nonneg (1-a^2)) (Real.sqrt_nonneg (1-b^2))
  nlinarith [sq_nonneg (c+a*b-Real.sqrt (1-a^2)*Real.sqrt (1-b^2))]
have hcosa : Real.cos (Real.arccos a) = a := Real.cos_arccos (by linarith) ha1
have hcosb : Real.cos (Real.arccos b) = b := Real.cos_arccos (by linarith) hb1
have hcos : -Real.cos (Real.arccos a + Real.arccos b) ≤ c := by
  rw [Real.cos_add, hcosa, hcosb, Real.sin_arccos, Real.sin_arccos]
  linarith
have hsum0 : 0 ≤ Real.arccos a + Real.arccos b :=
  add_nonneg (Real.arccos_nonneg _) (Real.arccos_nonneg _)
have hsum : Real.arccos a + Real.arccos b ≤ Real.pi := by
  linarith [Real.arccos_le_pi_div_two.mpr ha, Real.arccos_le_pi_div_two.mpr hb]
have hc' := Real.arccos_le_arccos hcos
rw [Real.arccos_neg, Real.arccos_cos hsum0 hsum] at hc'
linarith

/-- C1 with N=3: all real unit vectors, no finite grid restriction. -/
theorem c1_n3 {x y z : Vec2} (hx : unit2 x) (hy : unit2 y) (hz : unit2 z) :
  angle2 x y + angle2 x z + angle2 y z ≤ Real.pi := by
have hg := gram2 x y z
change dot2 x x = 1 at hx
change dot2 y y = 1 at hy
change dot2 z z = 1 at hz

```

```

rw [hx, hy, hz] at hg
have hp : -(dot2 x y * dot2 x z * dot2 y z) ≤
  |dot2 x y| * |dot2 x z| * |dot2 y z| := by
  calc
    _ ≤ |dot2 x y * dot2 x z * dot2 y z| := neg_le_abs _
    _ = _ := by simp only [abs_mul]
apply arccos_triple (abs_nonneg _) (abs_nonneg _) (abs_nonneg _)
  (dot2_bound hx hy) (dot2_bound hx hz) (dot2_bound hy hz)
simp only [sq_abs]
nlinarith [hg, hp]

/-- C1, N=4, obtained by adding the four three-line inequalities. -/
theorem c1_n4 {w x y z : Vec2} (hw : unit2 w) (hx : unit2 x)
  (hy : unit2 y) (hz : unit2 z) :
  angle2 w x + angle2 w y + angle2 w z + angle2 x y + angle2 x z + angle2 y z ≤
  2 * Real.pi := by
  linarith [c1_n3 hw hx hy, c1_n3 hw hx hz, c1_n3 hw hy hz, c1_n3 hx hy hz]

/-- C2, m=2, in R¹. This also supplies C3, d=1. -/
theorem c2_m2 {x y : ℝ} (hx : x*x=1) (hy : y*y=1) :
  Real.arccos |x*y| = 0 := by
  have hs : (x*y)^2=1 := by
    calc
      (x*y)^2 = (x*x)*(y*y) := by ring
      _ = 1 := by rw [hx, hy]; norm_num
  have hab : |x*y|=1 := by
    have := sq_abs (x*y)
    have := abs_nonneg (x*y)
  nlinarith
  rw [hab, Real.arccos_one]

/-- C3, d=2, is the three-line planar case. -/
theorem c3_d2 {x y z : Vec2} (hx : unit2 x) (hy : unit2 y) (hz : unit2 z) :
  angle2 x y + angle2 x z + angle2 y z ≤ Real.pi := c1_n3 hx hy hz

/-- C5, d=2, is the four-line planar case. -/
theorem c5_d2 {w x y z : Vec2} (hw : unit2 w) (hx : unit2 x)
  (hy : unit2 y) (hz : unit2 z) :
  angle2 w x + angle2 w y + angle2 w z + angle2 x y + angle2 x z + angle2 y z ≤
  2 * Real.pi := c1_n4 hw hx hy hz

#print axioms c2_m2
#print axioms c1_n4
#print axioms c3_d2
#print axioms c5_d2
#print axioms c2_m3
#print axioms c1_n3
end Angles

```

1.3.2 2. C4Pentagon.lean

[Pinned source file](#) · SHA-256 092e302e13dcc62784280594511a29a058f6b8191c56fac98ae09a18e33f964d

```

import Mathlib.Analysis.SpecialFunctions.Trigonometric.Inverse
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Ring
import Mathlib.Tactic.NormNum

/-- A scalar pentagon inequality for acute angles. -/
namespace C4Pentagon

```

```

private theorem sqrt_one_sub_sq_ge_one_sub {a : ℝ} (ha0 : 0 ≤ a) (ha1 : a ≤ 1) :
  1 - a ≤ Real.sqrt (1 - a ^ 2) := by
  have hsq : (Real.sqrt (1 - a ^ 2)) ^ 2 = 1 - a ^ 2 :=
    Real.sq_sqrt (by nlinarith)
  have hs := Real.sqrt_nonneg (1 - a ^ 2)
  nlinarith [mul_nonneg ha0 (sub_nonneg.mpr ha1)]

/-- The scalar estimate needed to close the acute C4 pentagon argument. -/
theorem pentagon_scalar {a b : ℝ} (ha0 : 0 ≤ a) (ha1 : a ≤ 1)
  (hb0 : 0 ≤ b) (hb1 : b ≤ 1) :
  Real.arccos (a * b) +
    Real.arcsin ((Real.sqrt (1 - a ^ 2) * Real.sqrt (1 - b ^ 2)) / (1 + a * b)) ≤
    Real.arccos a + Real.arccos b := by
  let s := Real.sqrt (1 - a ^ 2)
  let t := Real.sqrt (1 - b ^ 2)
  let r := s * t
  let u := a * b
  let d := 1 + u
  let L := Real.sqrt (1 - u ^ 2)
  have hs0 : 0 ≤ s := Real.sqrt_nonneg _
  have ht0 : 0 ≤ t := Real.sqrt_nonneg _
  have hs2 : s ^ 2 = 1 - a ^ 2 := Real.sq_sqrt (by nlinarith)
  have ht2 : t ^ 2 = 1 - b ^ 2 := Real.sq_sqrt (by nlinarith)
  have hr0 : 0 ≤ r := mul_nonneg hs0 ht0
  have hu0 : 0 ≤ u := mul_nonneg ha0 hb0
  have hu1 : u ≤ 1 := by nlinarith [mul_nonneg (sub_nonneg.mpr ha1) hb0]
  have hdpos : 0 < d := by dsimp [d]; linarith
  have hL0 : 0 ≤ L := Real.sqrt_nonneg _
  have hL2 : L ^ 2 = 1 - u ^ 2 := Real.sq_sqrt (by nlinarith)
  have hL1 : L ≤ 1 := by nlinarith
  have hr2 : r ^ 2 = (1 - a ^ 2) * (1 - b ^ 2) := by
    dsimp [r]
    rw [mul_pow, hs2, ht2]
  have hidentity : d ^ 2 - r ^ 2 = (a + b) ^ 2 := by
    dsimp [d, u]
    rw [hr2]
    ring
  have hab0 : 0 ≤ a + b := by linarith
  have hrd : r ≤ d := by nlinarith [sq_nonneg (a + b)]
  have hratio0 : 0 ≤ r / d := div_nonneg hr0 hdpos.le
  have hratio1 : r / d ≤ 1 := (div_le_one hdpos).mpr hrd
  have hcosE : Real.cos (Real.arcsin (r / d)) = (a + b) / d := by
    rw [Real.cos_arcsin]
  have hrad : 0 ≤ 1 - (r / d) ^ 2 := by nlinarith [sq_nonneg (r / d - 1)]
  have hsq : (Real.sqrt (1 - (r / d) ^ 2)) ^ 2 = ((a + b) / d) ^ 2 := by
    rw [Real.sq_sqrt hrad]
  calc
    1 - (r / d) ^ 2 = (d ^ 2 - r ^ 2) / d ^ 2 := by field_simp
    _ = ((a + b) / d) ^ 2 := by rw [hidentity, div_pow]
  nlinarith only [hsq, Real.sqrt_nonneg (1 - (r / d) ^ 2),
    div_nonneg hab0 hdpos.le]
  have hrbound : (1 - a) * (1 - b) ≤ r := by
    have hsa : 1 - a ≤ s := sqrt_one_sub_sq_ge_one_sub ha0 ha1
    have htb : 1 - b ≤ t := sqrt_one_sub_sq_ge_one_sub hb0 hb1
    have h1a : 0 ≤ 1 - a := by linarith
    have h1b : 0 ≤ 1 - b := by linarith
    exact (mul_le_mul hsa htb h1a h1b hs0).trans_eq rfl
  have hcosc : Real.cos (Real.arccos u) = u := Real.cos_arccos (by linarith) hu1
  have hcospa : Real.cos (Real.arccos a) = a := Real.cos_arccos (by linarith) ha1
  have hcospb : Real.cos (Real.arccos b) = b := Real.cos_arccos (by linarith) hb1
  have hsinc : Real.sin (Real.arccos u) = L := Real.sin_arccos u
  have hsinpa : Real.sin (Real.arccos a) = s := Real.sin_arccos a

```

```

have hsinqb : Real.sin (Real.arccos b) = t := Real.sin_arccos b
have hsinE : Real.sin (Real.arcsin (r / d)) = r / d :=
  Real.sin_arcsin (by linarith) hratio1
have hcosineq :
  Real.cos (Real.arccos a + Real.arccos b) ≤
  Real.cos (Real.arccos u + Real.arcsin (r / d)) := by
  rw [Real.cos_add, Real.cos_add, hcospa, hcosqb, hsinpa, hsinqb,
    hcosc, hsinc, hcosE, hsinE]
  have hfactor : u * (1 - a) * (1 - b) ≤ u * r := by
    calc
      u * (1 - a) * (1 - b) = u * ((1 - a) * (1 - b)) := by ring
      _ ≤ u * r := mul_le_mul_of_nonneg_left hrbound hu0
  have hL : L * r ≤ r := by simpa using mul_le_mul_of_nonneg_right hL1 hr0
  have htarget : (u - r) * d ≤ u * (a + b) - L * r := by
    dsimp [d]
    nlinarith only [hfactor, hL, hu0, hr0]
  change u - r ≤ u * ((a + b) / d) - L * (r / d)
  calc
    u - r ≤ (u * (a + b) - L * r) / d := (le_div_iff0 hdpos).2 htarget
    _ = u * ((a + b) / d) - L * (r / d) := by ring
have hleft0 : 0 ≤ Real.arccos u + Real.arcsin (r / d) :=
  add_nonneg (Real.arccos_nonneg _) (Real.arcsin_nonneg.mpr hratio0)
have hleftpi : Real.arccos u + Real.arcsin (r / d) ≤ Real.pi := by
  linarith [Real.arccos_le_pi_div_two.mpr hu0, Real.arcsin_le_pi_div_two (r / d)]
have hright0 : 0 ≤ Real.arccos a + Real.arccos b :=
  add_nonneg (Real.arccos_nonneg _) (Real.arccos_nonneg _)
have hrightpi : Real.arccos a + Real.arccos b ≤ Real.pi := by
  linarith [Real.arccos_le_pi_div_two.mpr ha0, Real.arccos_le_pi_div_two.mpr hb0]
have hangle : Real.arccos u + Real.arcsin (r / d) ≤
  Real.arccos a + Real.arccos b := by
  by_contra hn
  have hstrict := Real.strictAntiOn_cos ⟨hright0, hrightpi⟩
  ⟨hleft0, hleftpi⟩ (lt_of_not_ge hn)
  exact (not_lt_of_ge hcosineq) hstrict
simpa only [s, t, r, u, d] using hangle

/-- Angle form of 'pentagon_scalar' for angles in the first quadrant. -/
theorem pentagon_scalar_angles {p q : ℝ} (hp0 : 0 ≤ p) (hp1 : p ≤ Real.pi / 2)
  (hq0 : 0 ≤ q) (hq1 : q ≤ Real.pi / 2) :
  Real.arccos (Real.cos p * Real.cos q) +
  Real.arcsin ((Real.sin p * Real.sin q) / (1 + Real.cos p * Real.cos q)) ≤
  p + q := by
  have hcp : 0 ≤ Real.cos p := Real.cos_nonneg_of_mem_Icc ⟨by linarith, hp1⟩
  have hcq : 0 ≤ Real.cos q := Real.cos_nonneg_of_mem_Icc ⟨by linarith, hq1⟩
  have hsp : 0 ≤ Real.sin p :=
    Real.sin_nonneg_of_nonneg_of_le_pi hp0 (by linarith [Real.pi_pos])
  have hsq : 0 ≤ Real.sin q :=
    Real.sin_nonneg_of_nonneg_of_le_pi hq0 (by linarith [Real.pi_pos])
  have hsp' : Real.sqrt (1 - Real.cos p ^ 2) = Real.sin p := by
    rw [show 1 - Real.cos p ^ 2 = Real.sin p ^ 2 by
      nlinarith [Real.sin_sq_add_cos_sq p], Real.sqrt_sq hsp]
  have hsq' : Real.sqrt (1 - Real.cos q ^ 2) = Real.sin q := by
    rw [show 1 - Real.cos q ^ 2 = Real.sin q ^ 2 by
      nlinarith [Real.sin_sq_add_cos_sq q], Real.sqrt_sq hsq]
  have h := pentagon_scalar hcp (Real.cos_le_one p) hcq (Real.cos_le_one q)
  rw [hsp', hsq', Real.arccos_cos hp0 (by linarith [Real.pi_pos]),
    Real.arccos_cos hq0 (by linarith [Real.pi_pos])] at h
  exact h

/-- The reverse three-angle certificate used for the six-cycle estimate. -/
theorem arccos_triple_ge_pi {x y z : ℝ} (hx0 : 0 ≤ x) (hx1 : x ≤ 1)
  (hy0 : 0 ≤ y) (hy1 : y ≤ 1) (hz0 : 0 ≤ z)
  (h : x ^ 2 + y ^ 2 + z ^ 2 + 2 * x * y * z ≤ 1) :

```

```

    Real.pi ≤ Real.arccos x + Real.arccos y + Real.arccos z := by
have hsx : (Real.sqrt (1 - x ^ 2)) ^ 2 = 1 - x ^ 2 :=
  Real.sq_sqrt (by nlinarith)
have hsy : (Real.sqrt (1 - y ^ 2)) ^ 2 = 1 - y ^ 2 :=
  Real.sq_sqrt (by nlinarith)
have hprod : (Real.sqrt (1 - x ^ 2) * Real.sqrt (1 - y ^ 2)) ^ 2 =
  (1 - x ^ 2) * (1 - y ^ 2) := by rw [mul_pow, hsx, hsy]
have hxy0 : 0 ≤ x * y + z := by positivity
have hroot0 : 0 ≤ Real.sqrt (1 - x ^ 2) * Real.sqrt (1 - y ^ 2) := by
  positivity
have hroot : x * y + z ≤ Real.sqrt (1 - x ^ 2) * Real.sqrt (1 - y ^ 2) := by
  nlinarith only [h, hprod, hxy0, hroot0]
have hcos : Real.cos (Real.arccos x + Real.arccos y) ≤ -z := by
  rw [Real.cos_add, Real.cos_arccos (by linarith) hx1,
    Real.cos_arccos (by linarith) hy1, Real.sin_arccos, Real.sin_arccos]
  linarith
have hsum0 : 0 ≤ Real.arccos x + Real.arccos y :=
  add_nonneg (Real.arccos_nonneg _) (Real.arccos_nonneg _)
have hsum : Real.arccos x + Real.arccos y ≤ Real.pi := by
  linarith [Real.arccos_le_pi_div_two.mpr hx0, Real.arccos_le_pi_div_two.mpr hy0]
have hc' := Real.arccos_le_arccos hcos
rw [Real.arccos_neg, Real.arccos_cos hsum0 hsum] at hc'
linarith

/-- A spherical right-triangle relation identifies the gap angle. -/
theorem gap_angle_add_projection {B U T : ℝ}
  (hB0 : 0 ≤ B) (hB1 : B ≤ Real.pi / 2)
  (hU0 : 0 ≤ U) (hU1 : U ≤ Real.pi / 2)
  (hT0 : 0 ≤ T) (hT1 : T ≤ Real.pi / 2)
  (hrel : Real.sin B * Real.sin T = Real.cos B * Real.sin U * Real.cos T) :
  Real.arcsin ((Real.sin B * Real.sin U) / (1 + Real.cos B * Real.cos U)) + T =
    Real.arccos (Real.cos T * Real.cos U) := by
  let a := Real.cos B
  let b := Real.cos U
  let r := Real.sin B * Real.sin U
  let d := 1 + a * b
  have ha0 : 0 ≤ a := Real.cos_nonneg_of_mem_Icc (by linarith, hB1)
  have hb0 : 0 ≤ b := Real.cos_nonneg_of_mem_Icc (by linarith, hU1)
  have ha1 : a ≤ 1 := Real.cos_le_one B
  have hb1 : b ≤ 1 := Real.cos_le_one U
  have hSB0 : 0 ≤ Real.sin B :=
    Real.sin_nonneg_of_nonneg_of_le_pi hB0 (by linarith [Real.pi_pos])
  have hSU0 : 0 ≤ Real.sin U :=
    Real.sin_nonneg_of_nonneg_of_le_pi hU0 (by linarith [Real.pi_pos])
  have hr0 : 0 ≤ r := mul_nonneg hSB0 hSU0
  have hdpos : 0 < d := by dsimp [d]; nlinarith [mul_nonneg ha0 hb0]
  have hBsqr : (Real.sin B) ^ 2 = 1 - a ^ 2 := by
    dsimp [a]
    nlinarith [Real.sin_sq_add_cos_sq B]
  have hUsqr : (Real.sin U) ^ 2 = 1 - b ^ 2 := by
    dsimp [b]
    nlinarith [Real.sin_sq_add_cos_sq U]
  have hr2 : r ^ 2 = (1 - a ^ 2) * (1 - b ^ 2) := by
    dsimp [r]
    rw [mul_pow, hBsqr, hUsqr]
  have hidentity : d ^ 2 - r ^ 2 = (a + b) ^ 2 := by
    dsimp [d]
    rw [hr2]
    ring
  have hrd : r ≤ d := by nlinarith [sq_nonneg (a + b)]
  have hratio0 : 0 ≤ r / d := div_nonneg hr0 hdpos.le
  have hratio1 : r / d ≤ 1 := (div_le_one hdpos).mpr hrd
  have hcosE : Real.cos (Real.arcsin (r / d)) = (a + b) / d := by

```

```

rw [Real.cos_arcsin]
have hrad :  $0 \leq 1 - (r / d) ^ 2$  := by nlinarith [sq_nonneg (r / d - 1)]
have hsq :  $(\text{Real.sqrt } (1 - (r / d) ^ 2)) ^ 2 = ((a + b) / d) ^ 2$  := by
  rw [Real.sq_sqrt hrad]
calc
   $1 - (r / d) ^ 2 = (d ^ 2 - r ^ 2) / d ^ 2$  := by field_simp
   $= ((a + b) / d) ^ 2$  := by rw [hidentity, div_pow]
nlinarith only [hsq, Real.sqrt_nonneg (1 - (r / d) ^ 2),
  div_nonneg (add_nonneg ha0 hb0) hdpos.le]
have hsinE :  $\text{Real.sin } (\text{Real.arcsin } (r / d)) = r / d$  :=
  Real.sin_arcsin (by linarith) hratio1
have hcos :  $\text{Real.cos } (\text{Real.arcsin } (r / d) + T) = \text{Real.cos } T * b$  := by
  rw [Real.cos_add, hcosE, hsinE]
have hrelMul := congrArg (fun v :  $\mathbb{R} \Rightarrow v * \text{Real.sin } U$ ) hrel
have hUtrig := congrArg (fun v :  $\mathbb{R} \Rightarrow a * \text{Real.cos } T * v$ )
  (Real.sin_sq_add_cos_sq U)
calc
   $(a + b) / d * \text{Real.cos } T - r / d * \text{Real.sin } T =$ 
   $(\text{Real.cos } T * (a + b) - \text{Real.sin } T * r) / d$  := by ring
   $= \text{Real.cos } T * b$  := by
    apply (div_eq_iff (ne_of_gt hdpos)).2
    dsimp [d, r, a, b] at *
    nlinarith only [hrelMul, hUtrig]
have hangle0 :  $0 \leq \text{Real.arcsin } (r / d) + T$  :=
  add_nonneg (Real.arcsin_nonneg.mpr hratio0) hT0
have hanglepi :  $\text{Real.arcsin } (r / d) + T \leq \text{Real.pi}$  := by
  linarith [Real.arcsin_le_pi_div_two (r / d)]
have htargt := Real.arccos_cos hangle0 hanglepi
rw [hcos] at htargt
simp only [r, d, a, b] using htargt.symm

/-- Companion scalar inequality for the C4 six-cycle projection. -/
theorem six_cycle_scalar {T U V :  $\mathbb{R}$ } (hT0 :  $0 \leq T$ ) (hT1 :  $T \leq \text{Real.pi} / 2$ )
  (hU0 :  $0 \leq U$ ) (hU1 :  $U \leq \text{Real.pi} / 2$ )
  (hV0 :  $0 \leq V$ ) (hV1 :  $V \leq \text{Real.pi} / 2$ ) :
   $\text{Real.arcsin } (\text{Real.sin } U * \text{Real.sin } V) \leq$ 
   $\text{Real.arccos } (\text{Real.cos } T * \text{Real.cos } U) +$ 
   $\text{Real.arccos } (\text{Real.sin } T * \text{Real.cos } V) - \text{Real.pi} / 2$  := by
  let x :=  $\text{Real.cos } T * \text{Real.cos } U$ 
  let y :=  $\text{Real.sin } T * \text{Real.cos } V$ 
  let z :=  $\text{Real.sin } U * \text{Real.sin } V$ 
  have hcT0 :  $0 \leq \text{Real.cos } T$  := Real.cos_nonneg_of_mem_Icc (by linarith, hT1)
  have hcU0 :  $0 \leq \text{Real.cos } U$  := Real.cos_nonneg_of_mem_Icc (by linarith, hU1)
  have hcV0 :  $0 \leq \text{Real.cos } V$  := Real.cos_nonneg_of_mem_Icc (by linarith, hV1)
  have hst0 :  $0 \leq \text{Real.sin } T$  := Real.sin_nonneg_of_nonneg_of_le_pi hT0 (by linarith [Real.pi_pos])
  have hsu0 :  $0 \leq \text{Real.sin } U$  := Real.sin_nonneg_of_nonneg_of_le_pi hU0 (by linarith [Real.pi_pos])
  have hsv0 :  $0 \leq \text{Real.sin } V$  := Real.sin_nonneg_of_nonneg_of_le_pi hV0 (by linarith [Real.pi_pos])
  have hx0 :  $0 \leq x$  := mul_nonneg hcT0 hcU0
  have hy0 :  $0 \leq y$  := mul_nonneg hst0 hcV0
  have hz0 :  $0 \leq z$  := mul_nonneg hsu0 hsv0
  have hx1 :  $x \leq 1$  := by nlinarith [mul_nonneg (sub_nonneg.mpr (Real.cos_le_one T)) hcU0,
    ↪ Real.cos_le_one U]
  have hy1 :  $y \leq 1$  := by nlinarith [mul_nonneg (sub_nonneg.mpr (Real.sin_le_one T)) hcV0,
    ↪ Real.cos_le_one V]
  have hz1 :  $z \leq 1$  := by nlinarith [mul_nonneg (sub_nonneg.mpr (Real.sin_le_one U)) hsv0,
    ↪ Real.sin_le_one V]
  have hT :  $(\text{Real.cos } T) ^ 2 + (\text{Real.sin } T) ^ 2 = 1$  := by
    nlinarith [Real.sin_sq_add_cos_sq T]
  have hU :  $(\text{Real.cos } U) ^ 2 + (\text{Real.sin } U) ^ 2 = 1$  := by
    nlinarith [Real.sin_sq_add_cos_sq U]
  have hV :  $(\text{Real.cos } V) ^ 2 + (\text{Real.sin } V) ^ 2 = 1$  := by
    nlinarith [Real.sin_sq_add_cos_sq V]
  have hcoeff :  $(\text{Real.sin } T) ^ 2 + (\text{Real.cos } T) ^ 2 * (\text{Real.sin } U) ^ 2 =$ 

```

```

    (Real.sin U) ^ 2 + (Real.sin T) ^ 2 * (Real.cos U) ^ 2 := by
have hcT : (Real.cos T) ^ 2 = 1 - (Real.sin T) ^ 2 := by linarith
have hcU : (Real.cos U) ^ 2 = 1 - (Real.sin U) ^ 2 := by linarith
rw [hcT, hcU]
ring
let w := Real.cos T * Real.sin U * Real.cos V -
    Real.sin T * Real.sin V * Real.cos U
have hidentity : x ^ 2 + y ^ 2 + z ^ 2 + 2 * x * y * z + w ^ 2 = 1 := by
calc
  x ^ 2 + y ^ 2 + z ^ 2 + 2 * x * y * z + w ^ 2 =
    (Real.cos T) ^ 2 * (Real.cos U) ^ 2 +
    (Real.cos V) ^ 2 * ((Real.sin T) ^ 2 + (Real.cos T) ^ 2 * (Real.sin U) ^ 2) +
    (Real.sin V) ^ 2 * ((Real.sin U) ^ 2 + (Real.sin T) ^ 2 * (Real.cos U) ^ 2) := by
dsimp [x, y, z, w]
ring
- = (Real.cos T) ^ 2 * (Real.cos U) ^ 2 +
    ((Real.cos V) ^ 2 + (Real.sin V) ^ 2) *
    ((Real.sin U) ^ 2 + (Real.sin T) ^ 2 * (Real.cos U) ^ 2) := by
rw [hcoeff]
ring
- = (Real.cos T) ^ 2 * (Real.cos U) ^ 2 +
    ((Real.sin U) ^ 2 + (Real.sin T) ^ 2 * (Real.cos U) ^ 2) := by rw [hV]; ring
- = ((Real.cos T) ^ 2 + (Real.sin T) ^ 2) * (Real.cos U) ^ 2 +
    (Real.sin U) ^ 2 := by ring
- = 1 := by rw [hT, one_mul, hU]
have hcT : x ^ 2 + y ^ 2 + z ^ 2 + 2 * x * y * z ≤ 1 := by
  nlinarith only [hidentity, sq_nonneg w]
have htri := arccos_triple_ge_pi hx0 hx1 hy0 hy1 hz0 hcT
rw [show Real.arccos z = Real.pi / 2 - Real.arcsin z from rfl] at htri
dsimp [x, y, z] at htri
linarith

/-- The local C4 projection estimate under the exact right-triangle relations. -/
theorem local_projection_inequality {T B C U V : ℝ}
  (hT0 : 0 ≤ T) (hT1 : T ≤ Real.pi / 2)
  (hB0 : 0 ≤ B) (hB1 : B ≤ Real.pi / 2)
  (hC0 : 0 ≤ C) (hC1 : C ≤ Real.pi / 2)
  (hU0 : 0 ≤ U) (hU1 : U ≤ Real.pi / 2)
  (hV0 : 0 ≤ V) (hV1 : V ≤ Real.pi / 2)
  (h1 : Real.sin B * Real.sin T = Real.cos B * Real.sin U * Real.cos T)
  (h2 : Real.sin C * Real.cos T = Real.cos C * Real.sin V * Real.sin T) :
  Real.arcsin (Real.sin U * Real.sin V) ≤
    (B + U - Real.arccos (Real.cos B * Real.cos U)) +
    (C + V - Real.arccos (Real.cos C * Real.cos V)) := by
have hgap1 := gap_angle_add_projection hB0 hB1 hU0 hU1 hT0 hT1 h1
have hT'0 : 0 ≤ Real.pi / 2 - T := by linarith
have hT'1 : Real.pi / 2 - T ≤ Real.pi / 2 := by linarith
have h2' : Real.sin C * Real.sin (Real.pi / 2 - T) =
  Real.cos C * Real.sin V * Real.cos (Real.pi / 2 - T) := by
  simp only [Real.sin_pi_div_two_sub, Real.cos_pi_div_two_sub] using h2
have hgap2 := gap_angle_add_projection hC0 hC1 hV0 hV1 hT'0 hT'1 h2'
rw [Real.cos_pi_div_two_sub] at hgap2
have hbound1 := pentagon_scalar_angles hB0 hB1 hU0 hU1
have hbound2 := pentagon_scalar_angles hC0 hC1 hV0 hV1
have hsix := six_cycle_scalar hT0 hT1 hU0 hU1 hV0 hV1
linarith

#print axioms pentagon_scalar
#print axioms pentagon_scalar_angles
#print axioms arccos_triple_ge_pi
#print axioms six_cycle_scalar
#print axioms gap_angle_add_projection
#print axioms local_projection_inequality

```

end C4Pentagon

1.3.3 3. C4Extremum.lean

[Pinned source file](#) · SHA-256 418cf7266191c243478469276cd84d95f054942949e478b8bc78d9eed9a33ba9

```
import Mathlib.Analysis.Convex.Function
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Inverse
import Mathlib.Tactic.FieldSimp
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Ring

namespace C4Extremum

/-- An interior maximum of a convex function forces both endpoint values to equal it. -/
theorem convex_max_endpoints {f : ℝ → ℝ} {l r : ℝ}
  (hl : l < 0) (hr : 0 < r) (hconv : ConvexOn ℝ (Set.Icc l r) f)
  (hmax : ∀ t ∈ Set.Icc l r, f t ≤ f 0) :
  f l = f 0 ∧ f r = f 0 := by
  have hd : 0 < r - l := by linarith
  let a : ℝ := r / (r - l)
  let b : ℝ := -l / (r - l)
  have ha : 0 < a := div_pos hr hd
  have hb : 0 < b := div_pos (neg_pos.mpr hl) hd
  have hab : a + b = 1 := by
    dsimp [a, b]
    rw [← add_div]
    have hnum : r + -l = r - l := by ring
    rw [hnum]
    exact div_self (ne_of_gt hd)
  have hzero : a • l + b • r = (0 : ℝ) := by
    dsimp [a, b]
    field_simp [ne_of_gt hd]
    ring
  have hlin : l ∈ Set.Icc l r := ⟨le_rfl, by linarith⟩
  have hrin : r ∈ Set.Icc l r := ⟨by linarith, le_rfl⟩
  have hweighted := hconv.2 hlin hrin ha.le hb.le hab
  rw [hzero] at hweighted
  simp only [smul_eq_mul] at hweighted
  have hfl : f l ≤ f 0 := hmax l hlin
  have hfr : f r ≤ f 0 := hmax r hrin
  have hsum : a * f 0 + b * f 0 = f 0 := by
    calc
      a * f 0 + b * f 0 = (a + b) * f 0 := by ring
      _ = f 0 := by rw [hab]; ring
  constructor
  · apply le_antisymm hfl
    by_contra h
    have hlt : f l < f 0 := lt_of_not_ge h
    have hltmul : a * f l < a * f 0 := mul_lt_mul_of_pos_left hlt ha
    have hlemul : b * f r ≤ b * f 0 := mul_le_mul_of_nonneg_left hfr hb.le
    linarith
  · apply le_antisymm hfr
    by_contra h
    have hlt : f r < f 0 := lt_of_not_ge h
    have hltmul : b * f r < b * f 0 := mul_lt_mul_of_pos_left hlt hb
    have hlemul : a * f l ≤ a * f 0 := mul_le_mul_of_nonneg_left hfl ha.le
    linarith

/-- Abstract endpoint contradiction used after a rotation is shown to preserve constraints. -/
```

```

theorem rotation_endpoint_contradiction {α : Type*} (rotation : ℝ → α)
  (objective : α → ℝ) (complexity : α → ℕ) {x : α} {l r : ℝ}
  (hl : l < 0) (hr : 0 < r)
  (hconv : ConvexOn ℝ (Set.Icc l r) (objective ∘ rotation))
  (hzero : rotation 0 = x)
  (hmax : ∀ y, objective y ≤ objective x)
  (hselect : ∀ y, objective y = objective x → complexity y ≤ complexity x)
  (hgain : complexity x < complexity (rotation l) ∨
    complexity x < complexity (rotation r)) : False := by
  have hmax' : ∀ t ∈ Set.Icc l r,
    (objective ∘ rotation) t ≤ (objective ∘ rotation) 0 := by
    intro t _
    simpa only [Function.comp_apply, hzero] using hmax (rotation t)
  obtain ⟨hle, hre⟩ := convex_max_endpoints hl hr hconv hmax'
  rcases hgain with hgain | hgain
  • have heq : objective (rotation l) = objective x := by
    simpa only [Function.comp_apply, hzero] using hle
    exact (not_lt_of_ge (hselect (rotation l) heq)) hgain
  • have heq : objective (rotation r) = objective x := by
    simpa only [Function.comp_apply, hzero] using hre
    exact (not_lt_of_ge (hselect (rotation r) heq)) hgain

#print axioms convex_max_endpoints
#print axioms rotation_endpoint_contradiction

end C4Extremum

```

1.3.4 4. C4Rotation.lean

[Pinned source file](#) · SHA-256 af61ab1e5d9de2d496184a668e5bf66e8687b60d869f17ecf4ddf64685f8ae3a

```

import Mathlib.Analysis.Convex.Deriv
import Mathlib.Analysis.Normed.Module.Convex
import Mathlib.Analysis.SpecialFunctions.Trigonometric.InverseDeriv
import Mathlib.Analysis.SpecialFunctions.Sqrt
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Ring
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.FieldSimp

namespace C4Rotation

private theorem inner_sq_lt_one {r t : ℝ} (hr0 : 0 ≤ r) (hr1 : r < 1) :
  (r * Real.cos t) ^ 2 < 1 := by
  have hc := Real.abs_cos_le_one t
  have hcsq : (Real.cos t) ^ 2 ≤ 1 := by
    nlinarith [mul_nonneg (abs_nonneg (Real.cos t)) (sub_nonneg.mpr hc),
      sq_abs (Real.cos t)]
  have hr2 : r ^ 2 < 1 := by nlinarith
  nlinarith [mul_nonneg (sq_nonneg r) (sub_nonneg.mpr hcsq)]

private theorem denominator_pos {r t : ℝ} (hr0 : 0 ≤ r) (hr1 : r < 1) :
  0 < Real.sqrt (1 - (r * Real.cos t) ^ 2) := by
  exact Real.sqrt_pos.2 (by linarith [inner_sq_lt_one hr0 hr1 (t := t)])

theorem hasDerivAt_angle {r t : ℝ} (hr0 : 0 ≤ r) (hr1 : r < 1) :
  HasDerivAt (fun x : ℝ => Real.arccos (r * Real.cos x))
    (r * Real.sin t / Real.sqrt (1 - (r * Real.cos t) ^ 2)) t := by
  have hsq := inner_sq_lt_one hr0 hr1 (t := t)
  have hlt : r * Real.cos t < 1 := by nlinarith

```

```

have hgt : -1 < r * Real.cos t := by nlinarith
have houter := Real.hasDerivAt_arccos (by linarith : r * Real.cos t ≠ -1)
  (by linarith : r * Real.cos t ≠ 1)
have hinner : HasDerivAt (fun x : ℝ => r * Real.cos x) (r * -Real.sin t) t :=
  (Real.hasDerivAt_cos t).const_mul r
convert houter.comp t hinner using 1
· rfl
· simp only [div_eq_mul_inv]
  ring

theorem hasDerivAt_slope {r t : ℝ} (hr0 : 0 ≤ r) (hr1 : r < 1) :
  HasDerivAt
    (fun x : ℝ => r * Real.sin x / Real.sqrt (1 - (r * Real.cos x) ^ 2))
    (r * (1 - r ^ 2) * Real.cos t /
      ((1 - (r * Real.cos t) ^ 2) * Real.sqrt (1 - (r * Real.cos t) ^ 2))) t := by
have hcos : HasDerivAt (fun x : ℝ => r * Real.cos x) (r * -Real.sin t) t :=
  (Real.hasDerivAt_cos t).const_mul r
have hD : HasDerivAt (fun x : ℝ => 1 - (r * Real.cos x) ^ 2)
  (2 * r ^ 2 * Real.cos t * Real.sin t) t := by
  convert (hcos.pow 2).const_sub 1 using 1; ring
have hDpos : 0 < 1 - (r * Real.cos t) ^ 2 := by
  linarith [inner_sqrt_lt_one hr0 hr1 (t := t)]
have hsqrt : HasDerivAt
  (fun x : ℝ => Real.sqrt (1 - (r * Real.cos x) ^ 2))
  ((1 / (2 * Real.sqrt (1 - (r * Real.cos t) ^ 2))) *
    (2 * r ^ 2 * Real.cos t * Real.sin t)) t := by
  convert (Real.hasDerivAt_sqrt hDpos.ne').comp t hD using 1; rfl
have hnum : HasDerivAt (fun x : ℝ => r * Real.sin x)
  (r * Real.cos t) t := (Real.hasDerivAt_sin t).const_mul r
have hsqrt_ne : Real.sqrt (1 - (r * Real.cos t) ^ 2) ≠ 0 :=
  (denominator_pos hr0 hr1 (t := t)).ne'
convert hnum.div hsqrt hsqrt_ne using 1
have hsqrt2 : (Real.sqrt (1 - (r * Real.cos t) ^ 2)) ^ 2 =
  1 - (r * Real.cos t) ^ 2 := Real.sq_sqrt hDpos.le
have htrig := Real.sin_sq_add_cos_sq t
field_simp
ring_nf at hsqrt2
have hden : (1 - r ^ 2 * Real.cos t ^ 2) *
  Real.sqrt (1 - r ^ 2 * Real.cos t ^ 2) =
  Real.sqrt (1 - r ^ 2 * Real.cos t ^ 2) ^ 3 := by
  calc
    _ = (Real.sqrt (1 - r ^ 2 * Real.cos t ^ 2)) ^ 2 *
      Real.sqrt (1 - r ^ 2 * Real.cos t ^ 2) := by rw [hsqrt2]
    _ = _ := by ring
rw [hden]
congr 1
have hnumid : Real.sqrt (1 - r ^ 2 * Real.cos t ^ 2) ^ 2 -
  r ^ 2 * Real.sin t ^ 2 = 1 - r ^ 2 := by
  rw [hsqrt2]
  nlinarith [htrig]
rw [hnumid]
ring

private theorem convexOn_angle_lt_one {r : ℝ} (hr0 : 0 ≤ r) (hr1 : r < 1) :
  ConvexOn ℝ (Set.Icc (-(Real.pi / 2)) (Real.pi / 2))
  (fun t => Real.arccos (r * Real.cos t)) := by
let D : Set ℝ := Set.Icc (-(Real.pi / 2)) (Real.pi / 2)
have hD : Convex ℝ D := convex_Icc _ _
have hcont : ContinuousOn (fun t => Real.arccos (r * Real.cos t)) D :=
  (Real.continuous_arccos.comp (continuous_const.mul Real.continuous_cos)).continuousOn
apply convexOn_of_hasDerivWithinAt2_nonneg hD hcont
  (fun t _ => (hasDerivAt_angle hr0 hr1).hasDerivWithinAt)
  (fun t _ => (hasDerivAt_slope hr0 hr1).hasDerivWithinAt)

```

```

intro t ht
have htD : t ∈ D := interior_subset ht
have hcos : 0 ≤ Real.cos t := Real.cos_nonneg_of_mem_Icc htD
have hr2 : 0 ≤ 1 - r ^ 2 := by nlinarith
have hdp : 0 < 1 - (r * Real.cos t) ^ 2 := by
  linarith [inner_sq_lt_one hr0 hr1 (t := t)]
have hsp : 0 < Real.sqrt (1 - (r * Real.cos t) ^ 2) :=
  denominator_pos hr0 hr1
exact div_nonneg (mul_nonneg (mul_nonneg hr0 hr2) hcos)
  (mul_nonneg hdp.le hsp.le)

theorem convexOn_angle {r : ℝ} (hr0 : 0 ≤ r) (hr1 : r ≤ 1) :
  ConvexOn ℝ (Set.Icc (-(Real.pi / 2)) (Real.pi / 2))
    (fun t => Real.arccos (r * Real.cos t)) := by
  by_cases hr : r < 1
  · exact convexOn_angle_lt_one hr0 hr
  have hreq : r = 1 := by linarith
  subst r
  have hnorm : ConvexOn ℝ (Set.Icc (-(Real.pi / 2)) (Real.pi / 2))
    (fun t : ℝ => |t|) := by
    have hnorm0 : ConvexOn ℝ (Set.Icc (-(Real.pi / 2)) (Real.pi / 2))
      (norm : ℝ → ℝ) := convexOn_norm (convex_Icc _ _)
    convert hnorm0 using 1
  apply hnorm.congr
  intro t ht
  rcases ht with ⟨hl, hu⟩
  simp only [one_mul]
  rcases le_total 0 t with hnonneg | hnonpos
  · rw [abs_of_nonneg hnonneg, Real.arccos_cos hnonneg]
    linarith [Real.pi_pos]
  · have hneg : 0 ≤ -t := by linarith
    rw [abs_of_nonpos hnonpos, ← Real.cos_neg, Real.arccos_cos hneg]
    linarith [Real.pi_pos]

#print axioms C4Rotation.convexOn_angle

end C4Rotation

```

1.3.5 5. C4SparseStep.lean

[Pinned source file](#) · SHA-256 072592200ed4d8059b3a1538911a0e2c7dba50ec4ff738ea105db497cee02272

```

import Mathlib.Analysis.SpecialFunctions.Trigonometric.Inverse
import Mathlib.Analysis.Convex.Function
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Ring
import Mathlib.Tactic.FieldSimp
import Mathlib.Data.Finset.Max
import C4Extremum
import C4Rotation

namespace C4SparseStep

/-- A positive initial inner product determines a cosine phase whose positive
semicircle contains the initial configuration strictly in its interior. -/
theorem cosine_phase {a b : ℝ} (ha : 0 < a) (hab : a ^ 2 + b ^ 2 ≤ 1) :
  ∃ r p : ℝ, 0 < r ∧ r ≤ 1 ∧ -(Real.pi / 2) < p ∧ p < Real.pi / 2 ∧
    ∀ t : ℝ, a * Real.cos t + b * Real.sin t = r * Real.cos (t - p) := by
  let r := Real.sqrt (a ^ 2 + b ^ 2)
  have hrad : 0 < a ^ 2 + b ^ 2 := by positivity

```

```

have hr : 0 < r := Real.sqrt_pos.mpr hrad
have hr2 : r ^ 2 = a ^ 2 + b ^ 2 := Real.sq_sqrt hrad.le
have hr1 : r ≤ 1 := by nlinarith
have hb10 : -r < b := by nlinarith [sq_pos_of_pos ha]
have hb11 : b < r := by nlinarith [sq_pos_of_pos ha]
have hratio0 : -1 < b / r := by
  apply (lt_div_iff0 hr).2
  linarith
have hratio1 : b / r < 1 := (div_lt_one hr).2 hb11
let p := Real.arcsin (b / r)
have hp0 : -(Real.pi / 2) < p := Real.neg_pi_div_two_lt_arcsin.mpr hratio0
have hp1 : p < Real.pi / 2 := Real.arcsin_lt_pi_div_two.mpr hratio1
have hsin : Real.sin p = b / r := Real.sin_arcsin hratio0.le hratio1.le
have hcos : Real.cos p = a / r := by
  have hcos0 : 0 ≤ Real.cos p := Real.cos_arcsin_nonneg _
  have hsquare : (Real.cos p) ^ 2 = (a / r) ^ 2 := by
    have hid := Real.sin_sq_add_cos_sq p
    rw [hsin] at hid
    have hrat : (a / r) ^ 2 + (b / r) ^ 2 = 1 := by
      field_simp
      nlinarith only [hr2]
    nlinarith only [hid, hrat]
  nlinarith only [hsquare, hcos0, div_pos ha hr]
refine ⟨r, p, hr, hr1, hp0, hp1, ?_⟩
intro t
rw [Real.cos_sub, hcos, hsin]
field_simp

/-- A finite nonempty family of positive cosine arcs has a common closed
interval around zero, and one arc vanishes at its left endpoint. -/
theorem common_positive_segment {ι : Type*} [Fintype ι] [Nonempty ι]
  (p : ι → ℝ) (hp : ∀ i, -(Real.pi / 2) < p i ∧ p i < Real.pi / 2) :
  ∃ l u : ℝ, l < 0 ∧ 0 < u ∧
    (∀ i t, t ∈ Set.Icc l u → t - p i ∈ Set.Icc (-(Real.pi / 2)) (Real.pi / 2)) ∧
    ∃ i, Real.cos (l - p i) = 0 := by
  obtain ⟨imax, _, hmax⟩ := Finset.exists_max_image Finset.univ p Finset.univ_nonempty
  obtain ⟨imin, _, hmin⟩ := Finset.exists_min_image Finset.univ p Finset.univ_nonempty
  refine ⟨p imax - Real.pi / 2, p imin + Real.pi / 2, by linarith [(hp imax).2],
    by linarith [(hp imin).1], ?_, ?_⟩
  · intro i t ht
    have hi0 := hmax i (Finset.mem_univ i)
    have hi1 := hmin i (Finset.mem_univ i)
    constructor <=> linarith [ht.1, ht.2]
  · refine ⟨imax, ?_⟩
    rw [show p imax - Real.pi / 2 - p imax = -(Real.pi / 2) by ring,
      Real.cos_neg, Real.cos_pi_div_two]

/-- The finite-family part of the first-zero rotation argument. The individual
arc convexity hypothesis is discharged by the analytic rotation theorem. -/
theorem phase_family_endpoint {ι : Type*} [Fintype ι] [Nonempty ι]
  (r p : ι → ℝ) (hr : ∀ i, 0 ≤ r i)
  (hp : ∀ i, -(Real.pi / 2) < p i ∧ p i < Real.pi / 2)
  (hconv : ∀ i, ConvexOn ℝ (Set.Icc (-(Real.pi / 2)) (Real.pi / 2))
    (fun t => Real.arccos (r i * Real.cos t)))
  (hmax : ∀ t : ℝ,
    (∑ i, Real.arccos |r i * Real.cos (t - p i)|) ≤
    ∑ i, Real.arccos |r i * Real.cos (0 - p i)|) :
  ∃ t : ℝ,
    (∑ i, Real.arccos |r i * Real.cos (t - p i)|) =
    (∑ i, Real.arccos |r i * Real.cos (0 - p i)|) ∧
    ∃ i, r i * Real.cos (t - p i) = 0 := by
  obtain ⟨l, u, hl, hu, hseg, izero, hzero⟩ := common_positive_segment p hp
  let f : ℝ → ℝ := fun t => ∑ i, Real.arccos |r i * Real.cos (t - p i)|

```

```

have heach (i : ι) : ConvexOn ℝ (Set.Icc l u)
  (fun t => Real.arccos |r i * Real.cos (t - p i)|) := by
  have hsub : Set.Icc l u ⊆
    (fun t => -p i + t) ⁻¹' Set.Icc (-(Real.pi / 2)) (Real.pi / 2) := by
    intro t ht
    simp only [Set.mem_preimage, sub_eq_add_neg, add_comm] using hseg i t ht
  have hshift := ((hconv i).translate_right (-p i)).subset hsub (convex_Icc l u)
  apply hshift.congr
  intro t ht
  have hnonneg : 0 ≤ r i * Real.cos (t - p i) :=
    mul_nonneg (hr i) (Real.cos_nonneg_of_mem_Icc (hseg i t ht))
  change Real.arccos (r i * Real.cos (-p i + t)) =
    Real.arccos |r i * Real.cos (t - p i)|
  rw [show -p i + t = t - p i by ring, abs_of_nonneg hnonneg]
have hsum : ConvexOn ℝ (Set.Icc l u) f := by
  refine ⟨convex_Icc l u, ?_⟩
  intro x hx y hy a b ha hb hab
  change (∑ i, Real.arccos |r i * Real.cos (a * x + b * y - p i)|) ≤
    a * (∑ i, Real.arccos |r i * Real.cos (x - p i)|) +
    b * (∑ i, Real.arccos |r i * Real.cos (y - p i)|)
  calc
  - ≤ ∑ i, (a * Real.arccos |r i * Real.cos (x - p i)| +
    b * Real.arccos |r i * Real.cos (y - p i)|) := by
    apply Finset.sum_le_sum
    intro i _
    exact (heach i).2 hx hy ha hb hab
  - = _ := by rw [Finset.sum_add_distrib, Finset.mul_sum, Finset.mul_sum]
have hend := (C4Extremum.convex_max_endpoints hl hu hsum (fun t _ => hmax t)).1
exact {l, hend, izero, by rw [hzero, mul_zero]}

/-- A maximal finite sum of line angles can be rotated until one positive
initial inner product vanishes, without changing the sum. -/
theorem positive_coefficient_endpoint {ι : Type*} [Fintype ι] [Nonempty ι]
  (a b : ι → ℝ) (ha : ∀ i, 0 < a i)
  (hab : ∀ i, (a i) ^ 2 + (b i) ^ 2 ≤ 1)
  (hmax : ∀ t : ℝ,
    (∑ i, Real.arccos |a i * Real.cos t + b i * Real.sin t|) ≤
    ∑ i, Real.arccos |a i|) :
  ∃ t : ℝ,
    (∑ i, Real.arccos |a i * Real.cos t + b i * Real.sin t|) =
    (∑ i, Real.arccos |a i|) ∧
    ∃ i, a i * Real.cos t + b i * Real.sin t = 0 := by
  classical
  choose r p hr0 hr1 hp0 hp1 heq using fun i => cosine_phase (ha i) (hab i)
  have hsum (t : ℝ) :
    (∑ i, Real.arccos |r i * Real.cos (t - p i)|) =
    ∑ i, Real.arccos |a i * Real.cos t + b i * Real.sin t| := by
    apply Finset.sum_congr rfl
    intro i _
    rw [← heq i t]
  have hsum0 : (∑ i, Real.arccos |r i * Real.cos (0 - p i)|) =
    ∑ i, Real.arccos |a i| := by
    simp only [Real.cos_zero, Real.sin_zero, mul_one, mul_zero, add_zero] using hsum 0
  obtain ⟨t, ht, i, hi⟩ := phase_family_endpoint r p (fun i => (hr0 i).le)
  (fun i => ⟨hp0 i, hp1 i⟩) (fun i => C4Rotation.convexOn_angle (hr0 i).le (hr1 i))
  (fun t => by rw [hsum t, hsum0]; exact hmax t)
  refine ⟨t, ?_, i, ?_⟩
  · rwa [hsum t, hsum0] at ht
  · rwa [heq i t]

#print axioms cosine_phase
#print axioms common_positive_segment
#print axioms phase_family_endpoint

```

```
#print axioms positive_coefficient_endpoint
end C4SparseStep
```

1.3.6 6. C4SignedSparse.lean

[Pinned source file](#) · SHA-256 5bdcda7a554ed29ed6221743fe95a077ba52aba441239e0f6024bdc526db84a2

```
import C4SparseStep

namespace C4SignedSparse

/-- A maximizing rotation can create a new zero among the nonzero initial
coefficients; coefficients initially zero remain identically zero. -/
theorem coefficient_endpoint {ι : Type*} [Fintype ι]
  (a b : ι → ℝ) (hne : ∃ i, a i ≠ 0)
  (hzero : ∀ i, a i = 0 → b i = 0)
  (hab : ∀ i, a i ^ 2 + b i ^ 2 ≤ 1)
  (hmax : ∀ t : ℝ,
    (∑ i, Real.arccos |a i * Real.cos t + b i * Real.sin t|) ≤
    ∑ i, Real.arccos |a i|) :
  ∃ t : ℝ,
    (∑ i, Real.arccos |a i * Real.cos t + b i * Real.sin t|) =
    ∑ i, Real.arccos |a i| ∧
    (∀ i, a i = 0 → a i * Real.cos t + b i * Real.sin t = 0) ∧
    ∃ i, a i ≠ 0 ∧ a i * Real.cos t + b i * Real.sin t = 0 := by
  classical
  let J := {i : ι // a i ≠ 0}
  let K := {i : ι // ¬a i ≠ 0}
  have hJ : Nonempty J := by
    obtain ⟨i, hi⟩ := hne
    exact ⟨i, hi⟩
  let A : J → ℝ := fun j => |a j.1|
  let B : J → ℝ := fun j => if 0 < a j.1 then b j.1 else -b j.1
  have hA (j : J) : 0 < A j := abs_pos.mpr j.2
  have hAB (j : J) : A j ^ 2 + B j ^ 2 ≤ 1 := by
    by_cases hp : 0 < a j.1
    · simp [A, B, hp, abs_of_pos hp] using hab j.1
    · have hn : a j.1 < 0 := lt_of_le_of_ne (le_of_not_gt hp) j.2
      simp [A, B, hp, abs_of_neg hn] using hab j.1
  have hterm (j : J) (t : ℝ) :
    |A j * Real.cos t + B j * Real.sin t| =
    |a j.1 * Real.cos t + b j.1 * Real.sin t| := by
    by_cases hp : 0 < a j.1
    · simp [A, B, hp, abs_of_pos hp]
    · have hn : a j.1 < 0 := lt_of_le_of_ne (le_of_not_gt hp) j.2
      simp only [A, B, hp, ↓reduceIte, abs_of_neg hn]
      calc
        |-a j.1 * Real.cos t + -b j.1 * Real.sin t| =
        |-(a j.1 * Real.cos t + b j.1 * Real.sin t)| := by congr 1; ring
        _ = _ := abs_neg _
  let F : ℝ → ι → ℝ := fun t i => Real.arccos |a i * Real.cos t + b i * Real.sin t|
  have hsplit (t : ℝ) :
    (∑ i, F t i) = (∑ j : J, F t j.1) + ∑ k : K, F t k.1 := by
    exact (Fintype.sum_subtype_add_sum_subtype (fun i => a i ≠ 0) (F t)).symm
  have hK (t : ℝ) : (∑ k : K, F t k.1) = ∑ k : K, F 0 k.1 := by
    apply Finset.sum_congr rfl
    intro k _
    have ha : a k.1 = 0 := by simp using k.2
    simp [F, ha, hzero k.1 ha]
  have hmaxJ (t : ℝ) :
```

```

    (∑ j : J, Real.arccos |A j * Real.cos t + B j * Real.sin t|) ≤
    ∑ j : J, Real.arccos |A j| := by
  have htot : (∑ i, F t i) ≤ ∑ i, F 0 i := by
    simpa [F] using hmax t
  rw [hsplit t, hsplit 0, hK t] at htot
  have hJbound : (∑ j : J, F t j.1) ≤ ∑ j : J, F 0 j.1 := by linarith
  simpa [F, A, hterm] using hJbound
  have : Nonempty J := hJ
  obtain ⟨t, hsumJ, j, hj⟩ :=
    C4SparseStep.positive_coefficient_endpoint A B hA hAB hmaxJ
  refine ⟨t, ?_, ?_, j.1, j.2, ?_⟩
  · have hsumJ' : (∑ j : J, F t j.1) = ∑ j : J, F 0 j.1 := by
    simpa [F, A, hterm] using hsumJ
  calc
    (∑ i, Real.arccos |a i * Real.cos t + b i * Real.sin t|) =
      (∑ i, F t i) := rfl
    _ = (∑ j : J, F t j.1) + ∑ k : K, F t k.1 := hsplit t
    _ = (∑ j : J, F 0 j.1) + ∑ k : K, F 0 k.1 := by rw [hsumJ', hK]
    _ = ∑ i, F 0 i := (hsplit 0).symm
    _ = ∑ i, Real.arccos |a i| := by simp [F]
  · intro i hi
    simp [hi, hzero i hi]
  · by_cases hp : 0 < a j.1
    · simpa [A, B, hp, abs_of_pos hp] using hj
    · have hn : a j.1 < 0 := lt_of_le_of_ne (le_of_not_gt hp) j.2
      simp only [A, B, hp, ↓reduceIte, abs_of_neg hn] at hj
      nlinarith [hj]

#print axioms C4SignedSparse.coefficient_endpoint

end C4SignedSparse

```

1.3.7 7. C4Geometry.lean

[Pinned source file](#) · SHA-256 d08fae0bc5dd30adc77f62237269fd36f9e0c1fb2c3c56d0a0d48c5cd014dc4f

```

import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Inverse
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Ring

open scoped InnerProductSpace

namespace C4Geometry

variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]

/-- Rotation in the orthonormal plane spanned by `x` and `y`. -/
noncomputable def rotation (x y : E) (t : ℝ) : E := Real.cos t • x + Real.sin t • y

theorem inner_rotation (x y z : E) (t : ℝ) :
  ⟨rotation x y t, z⟩_R = ⟨x, z⟩_R * Real.cos t + ⟨y, z⟩_R * Real.sin t := by
  simp only [rotation, inner_add_left, real_inner_smul_left]
  ring

theorem rotation_preserves_orthogonality (x y z : E) (t : ℝ)
  (hxz : ⟨x, z⟩_R = 0) (hyz : ⟨y, z⟩_R = 0) :
  ⟨rotation x y t, z⟩_R = 0 := by
  rw [inner_rotation, hxz, hyz]
  ring

```

```

theorem rotation_unit (x y : E) (t : ℝ)
  (hx : ‖x‖ = 1) (hy : ‖y‖ = 1) (hxy : ⟨x, y⟩_R = 0) :
  ‖rotation x y t‖ = 1 := by
  have hxx : ⟨x, x⟩_R = 1 := by rw [real_inner_self_eq_norm_sq, hx]; norm_num
  have hyy : ⟨y, y⟩_R = 1 := by rw [real_inner_self_eq_norm_sq, hy]; norm_num
  have hxy : ⟨y, x⟩_R = 0 := by rw [real_inner_comm, hxy]
  have hrot : ⟨rotation x y t, rotation x y t⟩_R =
    (Real.cos t) ^ 2 + (Real.sin t) ^ 2 := by
    simp only [rotation, inner_add_left, inner_add_right,
      real_inner_smul_left, real_inner_smul_right, hxx, hyy, hxy, hxy]
    ring
  have hn : ‖rotation x y t‖ ^ 2 = 1 := by
    rw [← real_inner_self_eq_norm_sq, hrot]
    nlinarith [Real.sin_sq_add_cos_sq t]
    nlinarith [norm_nonneg (rotation x y t)]

theorem besse1_two (x y z : E)
  (hx : ‖x‖ = 1) (hy : ‖y‖ = 1) (hxy : ⟨x, y⟩_R = 0)
  (hz : ‖z‖ = 1) :
  ⟨x, z⟩_R ^ 2 + ⟨y, z⟩_R ^ 2 ≤ 1 := by
  let a : ℝ := ⟨x, z⟩_R
  let b : ℝ := ⟨y, z⟩_R
  let p : E := a • x + b • y
  have hxx : ⟨x, x⟩_R = 1 := by rw [real_inner_self_eq_norm_sq, hx]; norm_num
  have hyy : ⟨y, y⟩_R = 1 := by rw [real_inner_self_eq_norm_sq, hy]; norm_num
  have hxy : ⟨y, x⟩_R = 0 := by rw [real_inner_comm, hxy]
  have hzx : ⟨z, x⟩_R = a := by rw [real_inner_comm]
  have hzy : ⟨z, y⟩_R = b := by rw [real_inner_comm]
  have hpp : ⟨p, p⟩_R = a ^ 2 + b ^ 2 := by
    simp only [p, inner_add_left, inner_add_right,
      real_inner_smul_left, real_inner_smul_right, hxx, hyy, hxy, hxy]
    ring
  have hzp : ⟨z, p⟩_R = a ^ 2 + b ^ 2 := by
    simp only [p, inner_add_right, real_inner_smul_right, hzx, hzy]
    ring
  have hpn : ‖p‖ ^ 2 = a ^ 2 + b ^ 2 := by
    rw [← real_inner_self_eq_norm_sq]
    exact hpp
  have hres := norm_sub_sq_real z p
  rw [hz, hzp, hpn] at hres
  have hres0 : 0 ≤ ‖z - p‖ ^ 2 := sq_nonneg _
  dsimp [a, b] at *
  nlinarith

#print axioms inner_rotation
#print axioms rotation_preserves_orthogonality
#print axioms rotation_unit
#print axioms besse1_two

end C4Geometry

```

1.3.8 8. C4LocalRotation.lean

[Pinned source file](#) · SHA-256 e87a6df1b1713b3192a22afcedbc00bf210a0be7f51b50a4c3e2f655060635a0

```

import C4Geometry
import C4SignedSparse

open scoped InnerProductSpace

```

```

namespace C4LocalRotation

variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]

/-- If a line can move in an orthonormal plane while preserving every old
orthogonality, a maximal sum of its angles to a finite family can be preserved
while creating a new orthogonality. -/
theorem add_orthogonality {ι : Type*} [Fintype ι]
  (x y : E) (z : ι → E)
  (hx : ‖x‖ = 1) (hy : ‖y‖ = 1) (hxy : ⟨x, y⟩_ℝ = 0)
  (hz : ∀ i, ‖z i‖ = 1)
  (hactive : ∃ i, ⟨x, z i⟩_ℝ ≠ 0)
  (hpreserve : ∀ i, ⟨x, z i⟩_ℝ = 0 → ⟨y, z i⟩_ℝ = 0)
  (hmax : ∀ t : ℝ,
    (∑ i, Real.arccos |⟨C4Geometry.rotation x y t, z i⟩_ℝ|) ≤
    ∑ i, Real.arccos |⟨x, z i⟩_ℝ|) :
  ∃ t : ℝ, ‖C4Geometry.rotation x y t‖ = 1 ∧
    (∑ i, Real.arccos |⟨C4Geometry.rotation x y t, z i⟩_ℝ|) =
    (∑ i, Real.arccos |⟨x, z i⟩_ℝ|) ∧
    (∀ i, ⟨x, z i⟩_ℝ = 0 → ⟨C4Geometry.rotation x y t, z i⟩_ℝ = 0) ∧
    ∃ i, ⟨x, z i⟩_ℝ ≠ 0 ∧ ⟨C4Geometry.rotation x y t, z i⟩_ℝ = 0 := by
  have hcoeff : ∀ i, ⟨x, z i⟩_ℝ ^ 2 + ⟨y, z i⟩_ℝ ^ 2 ≤ 1 :=
    fun i => C4Geometry.bessel_two x y (z i) hx hy hxy (hz i)
  have hmax' : ∀ t : ℝ,
    (∑ i, Real.arccos |⟨x, z i⟩_ℝ * Real.cos t + ⟨y, z i⟩_ℝ * Real.sin t|) ≤
    ∑ i, Real.arccos |⟨x, z i⟩_ℝ| := by
    intro t
    simpa only [C4Geometry.inner_rotation] using hmax t
  obtain ⟨t, ht, hold, i, hi, hnew⟩ := C4SignedSparse.coefficient_endpoint
    (fun i => ⟨x, z i⟩_ℝ) (fun i => ⟨y, z i⟩_ℝ)
    hactive hpreserve hcoeff hmax'
  refine ⟨t, C4Geometry.rotation_unit x y t hx hy hxy, ?_, ?_, i, hi, ?_⟩
  · simpa only [C4Geometry.inner_rotation] using ht
  · intro j hj
    simpa only [C4Geometry.inner_rotation] using hold j hj
  · simpa only [C4Geometry.inner_rotation] using hnew

#print axioms add_orthogonality
end C4LocalRotation

```

1.3.9 9. C4Selection.lean

[Pinned source file](#) · SHA-256 768979c102b411b643bdf02eb47f8ef0b7535aeab151f745baaa00ae556e6720

```

import Mathlib.Topology.Order.Compact
import Mathlib.Order.Preorder.Finite
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Inverse

namespace C4Selection

/-- Choose a score maximizer with greatest natural-number complexity among all ties. -/
theorem compact_lex_max {α : Type*} [TopologicalSpace α]
  {K : Set α} {S : α → ℝ} {C : α → ℕ} {M : ℕ}
  (hK : IsCompact K) (hne : K.Nonempty) (hS : ContinuousOn S K)
  (hC : ∀ x ∈ K, C x ≤ M) :
  ∃ x ∈ K, (∀ y ∈ K, S y ≤ S x) ∧
    (∀ y ∈ K, S y = S x → C y ≤ C x) := by
  obtain ⟨x₀, hx₀, hmax⟩ := hK.exists_isMaxOn hne hS
  let T : Set α := {y | y ∈ K ∧ S y = S x₀}

```

```

have hTne : T.Nonempty := ⟨x₀, hx₀, rfl⟩
have hfinite : (C ' T).Finite := by
  apply (Set.finite_Icc 0 M).subset
  rintro n ⟨y, hy, rfl⟩
  exact ⟨Nat.zero_le _, hC y hy.1⟩
obtain ⟨x, hcx⟩ := Set.Finite.exists_maximalFor' C T hfinite hTne
have hx : x ∈ K ∧ S x = S x₀ := hcx.1
refine ⟨x, hx.1, ?_, ?_⟩
· intro y hy
  rw [hx.2]
  exact hmax hy
· intro y hy hscore
  have hyT : y ∈ T := ⟨hy, hscore.trans hx.2⟩
  rcases le_total (C y) (C x) with h | h
  · exact h
  · exact hcx.2 hyT h

#print axioms compact_lex_max

end C4Selection

```

1.3.10 10. C4Configuration.lean

[Pinned source file](#) · SHA-256 f43198c702add47aeb0477c1b85979a8b576e68b6c4cfd3035d6e155de3991d5

```

import C4Selection
import Mathlib.Analysis.InnerProductSpace.Continuous
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Inverse
import Mathlib.Topology.MetricSpace.ProperSpace

open scoped InnerProductSpace

namespace C4Configuration

variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]

abbrev UnitVector (E : Type*) [NormedAddCommGroup E] :=
  Metric.sphere (0 : E) 1

abbrev Configuration (E : Type*) [NormedAddCommGroup E] (n : ℕ) :=
  Fin n → UnitVector E

/-- Each unordered pair of indices appears once. -/
def pairs (n : ℕ) : Finset (Fin n × Fin n) :=
  Finset.univ.filter (fun p => p.1 < p.2)

noncomputable def totalAngle {n : ℕ} (v : Configuration E n) : ℝ :=
  ∑ p ∈ pairs n, Real.arccos |⟨(v p.1).1, (v p.2).1⟩_ℝ|

noncomputable def orthogonalityCount {n : ℕ} (v : Configuration E n) : ℕ :=
  ((pairs n).filter (fun p => ⟨(v p.1).1, (v p.2).1⟩_ℝ = 0)).card

theorem continuous_totalAngle (n : ℕ) :
  Continuous (totalAngle (E := E) (n := n)) := by
  classical
  unfold totalAngle
  apply continuous_finsetSum
  intro p _
  have hfirst : Continuous (fun v : Configuration E n => (v p.1).1) :=
    continuous_subtype_val.comp (continuous_apply p.1)

```

```

have hsecond : Continuous (fun v : Configuration E n => (v p.2).1) :=
  continuous_subtype_val.comp (continuous_apply p.2)
exact Real.continuous_arccos.comp ((hfirst.inner hsecond).abs)

theorem orthogonalityCount_le {n : ℕ} (v : Configuration E n) :
  orthogonalityCount v ≤ (pairs n).card := by
  classical
  exact Finset.card_filter_le _ _

/-- A geometric angle maximizer with the most orthogonal pairs among all ties. -/
theorem exists_lex_max {n : ℕ} [ProperSpace E] (u : UnitVector E) :
  ∃ v : Configuration E n,
    (∀ w : Configuration E n, totalAngle w ≤ totalAngle v) ∧
    (∀ w : Configuration E n, totalAngle w = totalAngle v →
      orthogonalityCount w ≤ orthogonalityCount v) := by
  have hne : (Set.univ : Set (Configuration E n)).Nonempty :=
    ⟨fun _ => u, Set.mem_univ _⟩
  obtain ⟨v, _, hmax, hselect⟩ :=
    C4Selection.compact_lex_max (K := Set.univ)
    (S := totalAngle (E := E) (n := n)) (C := orthogonalityCount)
    (M := (pairs n).card) isCompact_univ hne
    (continuous_totalAngle n).continuousOn
    (by intro w _; exact orthogonalityCount_le w)
  refine ⟨v, ?_, ?_⟩
  · intro w
    exact hmax w (Set.mem_univ w)
  · intro w hw
    exact hselect w (Set.mem_univ w) hw

#print axioms continuous_totalAngle
#print axioms orthogonalityCount_le
#print axioms exists_lex_max

end C4Configuration

```

1.3.11 11. C4Replacement.lean

[Pinned source file](#) · SHA-256 4d9286153ac302d4bf6d93e87418bffa41790380e034b574cc7667cd702222d3

```

import C4Configuration

open scoped InnerProductSpace

namespace C4Replacement

open C4Configuration

variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]

def replacement {n : ℕ} (v : Configuration E n) (i : Fin n)
  (w : UnitVector E) : Configuration E n := Function.update v i w

def pairInner {n : ℕ} (v : Configuration E n) (p : Fin n × Fin n) : ℝ :=
  ⟨(v p.1).1, (v p.2).1⟩_ℝ

noncomputable def pairAngle {n : ℕ} (v : Configuration E n) (p : Fin n × Fin n) : ℝ :=
  Real.arccos |pairInner v p|

def incidentPairs (n : ℕ) (i : Fin n) : Finset (Fin n × Fin n) :=
  (pairs n).filter (fun p => p.1 = i ∨ p.2 = i)

```

```
noncomputable def incidentAngle {n : ℕ} (v : Configuration E n) (i : Fin n) : ℝ :=
  ∑ p ∈ incidentPairs n i, pairAngle v p
```

```
theorem pairInner_replacement_nonincident {n : ℕ} (v : Configuration E n)
  (i : Fin n) (w : UnitVector E) (p : Fin n × Fin n)
  (h₁ : p.1 ≠ i) (h₂ : p.2 ≠ i) :
  pairInner (replacement v i w) p = pairInner v p := by
  simp [pairInner, replacement, Function.update_of_ne h₁, Function.update_of_ne h₂]
```

```
theorem pairAngle_replacement_nonincident {n : ℕ} (v : Configuration E n)
  (i : Fin n) (w : UnitVector E) (p : Fin n × Fin n)
  (h₁ : p.1 ≠ i) (h₂ : p.2 ≠ i) :
  pairAngle (replacement v i w) p = pairAngle v p := by
  rw [pairAngle, pairAngle, pairInner_replacement_nonincident v i w p h₁ h₂]
```

-- A replacement changes only the incident contribution to the total angle. --

```
theorem totalAngle_replacement_balance {n : ℕ} (v : Configuration E n)
  (i : Fin n) (w : UnitVector E) :
  totalAngle (replacement v i w) + incidentAngle v i =
  totalAngle v + incidentAngle (replacement v i w) i := by
  classical
  let q : Fin n × Fin n → Prop := fun p => p.1 = i ∨ p.2 = i
  have hother :
    ∑ p ∈ (pairs n).filter (fun p => ¬ q p), pairAngle (replacement v i w) p =
    ∑ p ∈ (pairs n).filter (fun p => ¬ q p), pairAngle v p := by
    apply Finset.sum_congr rfl
    intro p hp
    have hp' : ¬ q p := (Finset.mem_filter.mp hp).2
    have h₁ : p.1 ≠ i := fun h => hp' (Or.inl h)
    have h₂ : p.2 ≠ i := fun h => hp' (Or.inr h)
    exact pairAngle_replacement_nonincident v i w p h₁ h₂
  have hsplit (u : Configuration E n) :
    totalAngle u = incidentAngle u i +
    ∑ p ∈ (pairs n).filter (fun p => ¬ q p), pairAngle u p := by
    simp only [totalAngle, incidentAngle, incidentPairs, pairAngle]
    exact (Finset.sum_filter_add_sum_filter_not (pairs n) q
      (fun p => Real.arccos |pairInner u p|)).symm
  rw [hsplit (replacement v i w), hsplit v, hother]
  ring
```

```
theorem totalAngle_replacement_eq_of_incident_eq {n : ℕ}
  (v : Configuration E n) (i : Fin n) (w : UnitVector E)
  (h : incidentAngle (replacement v i w) i = incidentAngle v i) :
  totalAngle (replacement v i w) = totalAngle v := by
  have hb := totalAngle_replacement_balance v i w
  rw [h] at hb
  exact add_right_cancel hb
```

```
theorem incidentAngle_le_of_totalAngle_max {n : ℕ}
  (v : Configuration E n) (i : Fin n) (w : UnitVector E)
  (hmax : ∀ u : Configuration E n, totalAngle u ≤ totalAngle v) :
  incidentAngle (replacement v i w) i ≤ incidentAngle v i := by
  have hb := totalAngle_replacement_balance v i w
  have hm := hmax (replacement v i w)
  linarith
```

-- The uniquely ordered pair corresponding to two distinct indices. --

```
def edge {n : ℕ} (i j : Fin n) : Fin n × Fin n :=
  if i < j then (i, j) else (j, i)
```

```
theorem edge_mem_pairs {n : ℕ} {i j : Fin n} (hij : j ≠ i) :
  edge i j ∈ pairs n := by
```

```

have hne : i ≠ j := Ne.symm hij
rcases lt_or_gt_of_ne hne with h | h
· simp [edge, pairs, h]
· have hnot : ¬ i < j := not_lt.mpr h.le
  simp [edge, pairs, hnot, h]

theorem pairInner_edge {n : ℕ} (v : Configuration E n)
  (i j : Fin n) :
  pairInner v (edge i j) = ⟨⟨(v i).1, (v j).1⟩_R := by
by_cases h : i < j
· simp [pairInner, edge, h]
· simp [pairInner, edge, h, real_inner_comm]

theorem pairInner_replacement_edge {n : ℕ} (v : Configuration E n)
  (i j : Fin n) (w : UnitVector E) (hji : j ≠ i) :
  pairInner (replacement v i w) (edge i j) = ⟨w.1, (v j).1⟩_R := by
by_cases h : i < j
· simp [pairInner, edge, h, replacement, hji]
· simp [pairInner, edge, h, replacement, hji, real_inner_comm]

theorem edge_injective_away {n : ℕ} (i j k : Fin n)
  (hji : j ≠ i) (hki : k ≠ i)
  (h : edge i j = edge i k) : j = k := by
by_cases hj : i < j
· by_cases hk : i < k
  · simp [edge, hj, hk] using congrArg Prod.snd h
  · have hki' : k < i := (lt_or_gt_of_ne (Ne.symm hki)).resolve_left hk
    have hi : i = k := by simp [edge, hj, hk] using congrArg Prod.fst h
    exact (hki hi.symm).elim
· by_cases hk : i < k
  · have hji' : j < i := (lt_or_gt_of_ne (Ne.symm hji)).resolve_left hj
    have hi : j = i := by simp [edge, hj, hk] using congrArg Prod.fst h
    exact (hji hi).elim
  · simp [edge, hj, hk] using congrArg Prod.fst h

/-- The incident pair sum is the usual star sum over all other vertices. -/
theorem incidentAngle_eq_star {n : ℕ} (v : Configuration E n) (i : Fin n) :
  incidentAngle v i =
    ∑ j ∈ Finset.univ.erase i, Real.arccos |⟨(v i).1, (v j).1⟩_R| := by
classical
unfold incidentAngle
symm
apply Finset.sum_bij (fun j _ => edge i j)
· intro j hj
  have hji : j ≠ i := (Finset.mem_erase.mp hj).1
  apply Finset.mem_filter.mpr
  refine ⟨edge_mem_pairs hji, ?_⟩
  by_cases h : i < j
  · simp [edge, h]
  · simp [edge, h]
· intro j hj k hk heq
  exact edge_injective_away i j k (Finset.mem_erase.mp hj).1
  (Finset.mem_erase.mp hk).1 heq
· intro p hp
  obtain ⟨hpair, hinc⟩ := Finset.mem_filter.mp hp
  have hlt : p.1 < p.2 := (Finset.mem_filter.mp hpair).2
  rcases hinc with hleft | hright
  · refine ⟨p.2, Finset.mem_erase.mpr {?, Finset.mem_univ _}, ?_⟩
    · intro heq
      exact (ne_of_lt hlt) (hleft.trans heq.symm)
    · rcases p with ⟨a, b⟩
      simp only at hleft hlt
      subst a

```

```

      simp [edge, hlt]
    · refine ⟨p.1, Finset.mem_erase.mpr ⟨?, Finset.mem_univ _⟩, ?_⟩
    · intro heq
      exact (ne_of_lt hlt) (heq.trans hright.symm)
    · rcases p with ⟨a, b⟩
      simp only at hright hlt
      subst b
      have hnot : ¬ i < a := not_lt.mpr hlt.le
      simp [edge, hnot]
  · intro j hj
    rw [pairAngle, pairInner_edge]

/-- The incident sum indexed by neighbors as a subtype. -/
theorem incidentAngle_eq_sum_neighbors {n : ℕ}
  (v : Configuration E n) (i : Fin n) :
  incidentAngle v i =
    
$$\sum j : \{j : \text{Fin } n \mid j \neq i\}, \text{Real.arccos } |\langle (v i).1, (v j).1 \rangle_{\mathbb{R}}|$$

:= by
  classical
  rw [incidentAngle_eq_star]
  symm
  apply Finset.sum_bij (fun j _ => j.1)
  · intro j _
    exact Finset.mem_erase.mpr ⟨j.property, Finset.mem_univ _⟩
  · intro j _ k _ h
    exact Subtype.ext h
  · intro j hj
    exact ⟨⟨j, (Finset.mem_erase.mp hj).1⟩, Finset.mem_univ _, rfl⟩
  · intro j _
    rfl

/-- Exact replacement balance with the usual neighbor-index sums. -/
theorem totalAngle_replacement_star_balance {n : ℕ}
  (v : Configuration E n) (i : Fin n) (w : UnitVector E) :
  totalAngle (replacement v i w) +
    
$$\left( \sum j \in \text{Finset.univ.erase } i, \text{Real.arccos } |\langle (v i).1, (v j).1 \rangle_{\mathbb{R}}| \right) =$$

  totalAngle v +
    
$$\left( \sum j \in \text{Finset.univ.erase } i, \text{Real.arccos } |\langle w.1, (v j).1 \rangle_{\mathbb{R}}| \right)$$

:= by
  classical
  have hb := totalAngle_replacement_balance v i w
  rw [incidentAngle_eq_star v i,
    incidentAngle_eq_star (replacement v i w) i] at hb
  have hnewsum :
    
$$\left( \sum j \in \text{Finset.univ.erase } i, \text{Real.arccos } |\langle (\text{replacement } v \text{ i } w \text{ i}).1, \right.$$


$$\left. (\text{replacement } v \text{ i } w \text{ j}).1 \rangle_{\mathbb{R}}| \right) =$$


$$\left( \sum j \in \text{Finset.univ.erase } i, \text{Real.arccos } |\langle w.1, (v j).1 \rangle_{\mathbb{R}}| \right)$$

:= by
    apply Finset.sum_congr rfl
    intro j hj
    have hji : j ≠ i := (Finset.mem_erase.mp hj).1
    simp [replacement, hji]
  rw [hnewsum] at hb
  exact hb

theorem neighbor_sum_le_of_totalAngle_max {n : ℕ}
  (v : Configuration E n) (i : Fin n) (w : UnitVector E)
  (hmax : ∀ u : Configuration E n, totalAngle u ≤ totalAngle v) :
  
$$\left( \sum j : \{j : \text{Fin } n \mid j \neq i\}, \text{Real.arccos } |\langle w.1, (v j).1 \rangle_{\mathbb{R}}| \right) \leq$$


$$\left( \sum j : \{j : \text{Fin } n \mid j \neq i\},$$


```

```

    Real.arccos |⟨⟨(v i).1, (v j.1).1⟩_R|) := by
have h := incidentAngle_le_of_totalAngle_max v i w hmax
rw [incidentAngle_eq_sum_neighbors (replacement v i w) i,
    incidentAngle_eq_sum_neighbors v i] at h
have hnews :
  (∑ j : {j : Fin n // j ≠ i},
    Real.arccos |⟨⟨replacement v i w i).1,
      (replacement v i w j.1).1⟩_R|) =
  (∑ j : {j : Fin n // j ≠ i},
    Real.arccos |⟨w.1, (v j.1).1⟩_R|) := by
  apply Finset.sum_congr rfl
  intro j _
  simp [replacement, j.property]
rw [hnews] at h
exact h

theorem pairInner_zero_preserved {n : ℕ} (v : Configuration E n)
  (i : Fin n) (w : UnitVector E)
  (hpres : ∀ j : Fin n, j ≠ i →
    ⟨⟨(v i).1, (v j).1⟩_R = 0 → ⟨w.1, (v j).1⟩_R = 0)
  (p : Fin n × Fin n) (hp : p ∈ pairs n)
  (hzero : pairInner v p = 0) :
  pairInner (replacement v i w) p = 0 := by
by_cases hleft : p.1 = i
· have hright : p.2 ≠ i := by
  intro heq
  have hlt : p.1 < p.2 := (Finset.mem_filter.mp hp).2
  simp [hleft, heq] at hlt
  have hold : ⟨⟨(v i).1, (v p.2).1⟩_R = 0 := by
    simpa [pairInner, hleft] using hzero
  have hnew := hpres p.2 hright hold
  simpa [pairInner, replacement, hleft, hright] using hnew
· by_cases hright : p.2 = i
  · have hold : ⟨⟨(v i).1, (v p.1).1⟩_R = 0 := by
    rw [real_inner_comm]
    simpa [pairInner, hright] using hzero
    have hnew := hpres p.1 hleft hold
    rw [real_inner_comm] at hnew
    simpa [pairInner, replacement, hleft, hright] using hnew
  · rw [pairInner_replacement_nonincident v i w p hleft hright]
    exact hzero

/-- Preserving old incident zeros and creating one new zero increases the count. -/
theorem orthogonalityCount_replacement_strict {n : ℕ}
  (v : Configuration E n) (i : Fin n) (w : UnitVector E)
  (hpres : ∀ j : Fin n, j ≠ i →
    ⟨⟨(v i).1, (v j).1⟩_R = 0 → ⟨w.1, (v j).1⟩_R = 0)
  (hgain : ∃ j : Fin n, j ≠ i ∧
    ⟨⟨(v i).1, (v j).1⟩_R ≠ 0 ∧ ⟨w.1, (v j).1⟩_R = 0) :
  orthogonalityCount v < orthogonalityCount (replacement v i w) := by
classical
let old := (pairs n).filter (fun p => pairInner v p = 0)
let new := (pairs n).filter (fun p => pairInner (replacement v i w) p = 0)
have hsubset : old ⊆ new := by
  intro p hp
  obtain ⟨hpair, hzero⟩ := Finset.mem_filter.mp hp
  exact Finset.mem_filter.mpr
    ⟨hpair, pairInner_zero_preserved v i w hpres p hpair hzero⟩
obtain ⟨j, hji, hold, hnew⟩ := hgain
have hp : edge i j ∈ pairs n := edge_mem_pairs hji
have hpnew : edge i j ∈ new := by
  apply Finset.mem_filter.mpr
  constructor

```

```

    • exact hp
    • rw [pairInner_replacement_edge v i j w hji]
      exact hnew
have hpnot : edge i j  $\notin$  old := by
  intro h
  have hz : pairInner v (edge i j) = 0 := (Finset.mem_filter.mp h).2
  rw [pairInner_edge] at hz
  exact hold hz
have hstrict : old  $\subset$  new :=
  (Finset.ssubset_iff_of_subset hsubset).2 (edge i j, hpnew, hpnot)
exact Finset.card_lt_card hstrict

#print axioms totalAngle_replacement_balance
#print axioms totalAngle_replacement_eq_of_incident_eq
#print axioms incidentAngle_le_of_totalAngle_max
#print axioms incidentAngle_eq_star
#print axioms incidentAngle_eq_sum_neighbors
#print axioms totalAngle_replacement_star_balance
#print axioms neighbor_sum_le_of_totalAngle_max
#print axioms orthogonalityCount_replacement_strict

end C4Replacement

```

1.3.12 12. C4Direction.lean

[Pinned source file](#) · SHA-256 a1b1b936ccf804d516739f861176d80e25368529f39af3cdd8d18c564005c953

```

import Mathlib.Analysis.InnerProductSpace.Projection.FiniteDimensional
import Mathlib.LinearAlgebra.FiniteDimensional.Lemmas

open scoped InnerProductSpace

namespace C4Direction

/-- If a subspace and one extra vector leave room in a finite-dimensional
real inner-product space, choose a unit vector orthogonal to both. -/
theorem exists_unit_orthogonal {E : Type*} [NormedAddCommGroup E]
  [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
  (W : Submodule ℝ E) (x : E)
  (hdim : Module.finiteRank ℝ W + 1 < Module.finiteRank ℝ E) :
   $\exists y : E, \|y\| = 1 \wedge \langle x, y \rangle_{\mathbb{R}} = 0 \wedge$ 
     $\forall w \in W, \langle y, w \rangle_{\mathbb{R}} = 0$  := by
  let K : Submodule ℝ E := W  $\sqcup$  ℝ • x
  have hspan : Module.finiteRank ℝ (ℝ • x)  $\leq$  1 := by
    simp using (finiteRank_span_le_card ({x} : Set E))
  have hK : Module.finiteRank ℝ K  $\leq$  Module.finiteRank ℝ W + 1 := by
    have hsup := Submodule.finiteRank_sup_add_finiteRank_inf_eq W (ℝ • x)
    dsimp [K]
    omega
  have horth : 0 < Module.finiteRank ℝ K⊥ := by
    have hdimorth := K.finiteRank_add_finiteRank_orthogonal
    omega
  have hne : K⊥  $\neq$  ⊥ := by
    intro heq
    rw [heq, finiteRank_bot] at horth
    omega
  obtain ⟨z, hz, hz0⟩ := Submodule.exists_mem_ne_zero_of_ne_bot hne
  let y : E := ( $\|z\|^{-1}$  : ℝ) • z
  have hy : y ∈ K⊥ := Submodule.smul_mem _ _ hz
  have hynorm :  $\|y\| = 1$  := by

```

```

    change  $\|(\|z\|^{-1} : \mathbb{R}) \cdot z\| = 1$ 
    convert norm_smul_inv_norm ( $\mathbb{K} := \mathbb{R}$ ) hz0 using 1
  refine ⟨y, hynorm, ?_, ?_⟩
  • have hxK :  $x \in K := \text{Submodule.mem\_sup\_right } (\text{Submodule.mem\_span\_singleton\_self } x)$ 
    have h :=  $(K.\text{mem\_orthogonal}' y).\text{mp } h_y x hxK$ 
    rwa [real_inner_comm] at h
  • intro w hw
    exact  $(K.\text{mem\_orthogonal}' y).\text{mp } h_y w (\text{Submodule.mem\_sup\_left } hw)$ 

#print axioms C4Direction.exists_unit_orthogonal

end C4Direction

```

1.3.13 13. C4MaximalSparse.lean

[Pinned source file](#) · SHA-256 370d792a5e0d7b6b1b56dfbdf57577b68eac91b9690b8b328caaeb6019513218

```

import C4Configuration
import C4LocalRotation
import C4Direction
import C4Replacement
import Mathlib.LinearAlgebra.Span.Basic

open scoped InnerProductSpace

namespace C4MaximalSparse

open C4Configuration

variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]

omit [InnerProductSpace ℝ E] in
theorem unit_norm (x : UnitVector E) :  $\|x.1\| = 1$  := by
  simpa only [Metric.mem_sphere, dist_zero_right] using x.2

noncomputable def orthogonalNeighborSpan {n : ℕ} (v : Configuration E n) (i : Fin n) :
  Submodule ℝ E :=
  Submodule.span ℝ (Set.range (fun j : {j : Fin n //  $\langle\langle v.i \rangle\rangle.1, \langle\langle v.j \rangle\rangle.1 \rangle_{\mathbb{R}} = 0$ } =>
    (v.j.1).1))

theorem neighbor_mem_span {n : ℕ} (v : Configuration E n) (i j : Fin n)
  (h :  $\langle\langle v.i \rangle\rangle.1, \langle\langle v.j \rangle\rangle.1 \rangle_{\mathbb{R}} = 0$ ) :
  (v.j).1 ∈ orthogonalNeighborSpan v i := by
  apply Submodule.subset_span
  exact ⟨⟨j, h⟩, rfl⟩

private theorem replacement_incident {n : ℕ} (v : Configuration E n) (i : Fin n)
  (w : UnitVector E) :
  C4Replacement.incidentAngle (C4Replacement.replacement v i w) i =
   $\sum j : \{j : \text{Fin } n // j \neq i\}, \text{Real.arccos } |\langle\langle w.1, \langle\langle v.j \rangle\rangle.1 \rangle_{\mathbb{R}}|$  := by
  classical
  rw [C4Replacement.incidentAngle_eq_sum_neighbors]
  apply Finset.sum_congr rfl
  intro j _
  simp only [C4Replacement.replacement, Function.update_self,
    Function.update_of_ne j.property]

/-- At a maximizer with the largest orthogonality count, a vertex that has a
nonorthogonal neighbor has an orthogonal-neighbor span of codimension at most one. -/
theorem orthogonalNeighbor_finrank [FiniteDimensional ℝ E] {n : ℕ}

```

```

(v : Configuration E n) (i : Fin n)
(hmax : ∀ w : Configuration E n, totalAngle w ≤ totalAngle v)
(hselect : ∀ w : Configuration E n, totalAngle w = totalAngle v →
  orthogonalityCount w ≤ orthogonalityCount v)
(hactive : ∃ j : Fin n, j ≠ i ∧ ⟨(v i).1, (v j).1⟩_R ≠ 0) :
Module.finrank R E ≤ Module.finrank R (orthogonalNeighborSpan v i) + 1 := by
classical
by_contra hn
have hdim : Module.finrank R (orthogonalNeighborSpan v i) + 1 <
  Module.finrank R E := lt_of_not_ge hn
obtain ⟨y, hy, hxy, hyW⟩ := C4Direction.exists_unit_orthogonal
  (orthogonalNeighborSpan v i) (v i).1 hdim
let z : {j : Fin n // j ≠ i} → E := fun j => (v j.1).1
have hz : ∀ j, ‖z j‖ = 1 := fun j => unit_norm (v j.1)
have hactive' : ∃ j, ⟨(v i).1, z j⟩_R ≠ 0 := by
  obtain ⟨j, hji, hj⟩ := hactive
  exact ⟨⟨j, hji⟩, hj⟩
have hpreserve : ∀ j, ⟨(v i).1, z j⟩_R = 0 → ⟨y, z j⟩_R = 0 := by
  intro j hj
  exact hyW (z j) (neighbor_mem_span v i j.1 hj)
have hstarmax : ∀ t : R,
  (∑ j, Real.arccos |⟨C4Geometry.rotation (v i).1 y t, z j⟩_R|) ≤
    ∑ j, Real.arccos |⟨(v i).1, z j⟩_R| := by
  intro t
  let w : UnitVector E := ⟨C4Geometry.rotation (v i).1 y t, by
    simp only [Metric.mem_sphere, dist_zero_right] using
    C4Geometry.rotation_unit (v i).1 y t (unit_norm (v i)) hy hxy⟩
  have hb := C4Replacement.incidentAngle_le_of_totalAngle_max v i w hmax
  rw [replacement_incident, C4Replacement.incidentAngle_eq_sum_neighbors] at hb
  exact hb
obtain ⟨t, hunit, ht, hold, j, hj, hnew⟩ := C4LocalRotation.add_orthogonality
  (v i).1 y z (unit_norm (v i)) hy hxy hz hactive' hpreserve hstarmax
let w : UnitVector E := ⟨C4Geometry.rotation (v i).1 y t, by
  simp only [Metric.mem_sphere, dist_zero_right] using hunit⟩
have hincident : C4Replacement.incidentAngle (C4Replacement.replacement v i w) i =
  C4Replacement.incidentAngle v i := by
  rw [replacement_incident, C4Replacement.incidentAngle_eq_sum_neighbors]
  exact ht
have hscore : totalAngle (C4Replacement.replacement v i w) = totalAngle v :=
  C4Replacement.totalAngle_replacement_eq_of_incident_eq v i w hincident
have hstrict : orthogonalityCount v <
  orthogonalityCount (C4Replacement.replacement v i w) := by
  apply C4Replacement.orthogonalityCount_replacement_strict v i w
  · intro k hki hk
    exact hold ⟨k, hki⟩ hk
  · exact ⟨j.1, j.2, hj, hnew⟩
  exact (not_lt_of_ge (hselect (C4Replacement.replacement v i w) hscore)) hstrict

#print axioms neighbor_mem_span
#print axioms orthogonalNeighbor_finrank
end C4MaximalSparse

```

1.3.14 14. C4NeighborCount.lean

[Pinned source file](#) · SHA-256 9d4f9fa2f09f805192d2ac8b9d9a3e9801615b11b0f513fa83e0c1abb5d74833

```

import C4MaximalSparse
import Mathlib.LinearAlgebra.Dimension.Constructions
import Mathlib.Data.Finset.Card

```

```

open scoped InnerProductSpace

namespace C4NeighborCount

open C4Configuration C4MaximalSparse

variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]

noncomputable def nonorthogonalNeighbors {n : ℕ}
  (v : Configuration E n) (i : Fin n) : Finset (Fin n) :=
  Finset.univ.filter (fun j => j ≠ i ∧ ⟨(v i).1, (v j).1⟩_ℝ ≠ 0)

theorem neighbor_span_finrank_le_zero_count {n : ℕ}
  (v : Configuration E n) (i : Fin n) :
  Module.finrank ℝ (orthogonalNeighborSpan v i) ≤
  (Finset.univ.filter (fun j : Fin n =>
    ⟨(v i).1, (v j).1⟩_ℝ = 0)).card := by
  classical
  have hcard : Fintype.card {j : Fin n // ⟨(v i).1, (v j).1⟩_ℝ = 0} =
    (Finset.univ.filter (fun j : Fin n =>
      ⟨(v i).1, (v j).1⟩_ℝ = 0)).card := by
    rw [Fintype.card_subtype]
  have h := finrank_range_le_card (R := ℝ)
    (fun j : {j : Fin n // ⟨(v i).1, (v j).1⟩_ℝ = 0} => (v j.1).1)
  rw [hcard] at h
  convert h using 1
  rfl

theorem nonorthogonal_neighbors_le {n d : ℕ} (v : Configuration E n)
  (hcn : d ≤ n)
  (hrank : ∀ i : Fin n, (∃ j : Fin n,
    j ≠ i ∧ ⟨(v i).1, (v j).1⟩_ℝ ≠ 0) →
    d ≤ Module.finrank ℝ (orthogonalNeighborSpan v i) + 1)
  (i : Fin n) : (nonorthogonalNeighbors v i).card ≤ n - d := by
  classical
  let P : Fin n → Prop := fun j => ⟨(v i).1, (v j).1⟩_ℝ = 0
  let Z := Finset.univ.filter P
  let N := Finset.univ.filter (fun j => ¬ P j)
  have hself : ¬ P i := by
    dsimp [P]
    rw [real_inner_self_eq_norm_sq, unit_norm (v i)]
    norm_num
  have hiN : i ∈ N := by simp [N, hself]
  have hdegree : (nonorthogonalNeighbors v i).card + 1 = N.card := by
    have hset : nonorthogonalNeighbors v i = N.erase i := by
      ext j
      simp [nonorthogonalNeighbors, N, P, Finset.mem_erase]
    rw [hset]
    exact Finset.card_erase_add_one hiN
  have hpartition : Z.card + N.card = n := by
    simpa [Z, N, P] using
      (Finset.card_filter_add_card_filter_not (s := Finset.univ) P)
  by_cases hactive : ∃ j : Fin n,
    j ≠ i ∧ ⟨(v i).1, (v j).1⟩_ℝ ≠ 0
  • have hspan := neighbor_span_finrank_le_zero_count v i
    have hlower := hrank i hactive
    change Module.finrank ℝ (orthogonalNeighborSpan v i) ≤ Z.card at hspan
    omega
  • have hempty : nonorthogonalNeighbors v i = ∅ := by
    apply Finset.eq_empty_iff_forall_notMem.mpr
    intro j hj
    exact hactive (j, (Finset.mem_filter.mp hj).2)
    simp [hempty]

```

```
#print axioms C4NeighborCount.neighbor_span_finrank_le_zero_count
#print axioms C4NeighborCount.nonorthogonal_neighbors_le

end C4NeighborCount
```

1.3.15 15. C4SparseConfiguration.lean

[Pinned source file](#) · SHA-256 7bd4a7aa9e3b8b48313688d6e6126ab219cf60c54ad275089fe8e9fd050f9932

```
import C4NeighborCount

namespace C4SparseConfiguration

open C4Configuration

variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  [FiniteDimensional ℝ E] [ProperSpace E]

/-- An angle-maximizing configuration can be chosen so that each line has at
most `n - dim E` nonorthogonal neighbors. -/
theorem exists_sparse_maximizer {n : ℕ} (hdim : Module.finrank ℝ E ≤ n)
  (u : UnitVector E) :
  ∃ v : Configuration E n,
    (∀ w : Configuration E n, totalAngle w ≤ totalAngle v) ∧
    (∀ w : Configuration E n, totalAngle w = totalAngle v →
      orthogonalityCount w ≤ orthogonalityCount v) ∧
    ∀ i : Fin n, (C4NeighborCount.nonorthogonalNeighbors v i).card ≤
      n - Module.finrank ℝ E := by
  obtain ⟨v, hmax, hselect⟩ := exists_lex_max (n := n) u
  refine ⟨v, hmax, hselect, ?_⟩
  exact C4NeighborCount.nonorthogonal_neighbors_le v hdim
    (fun i hi => C4MaximalSparse.orthogonalNeighbor_finrank v i hmax hselect hi)

/-- The C4 reduction to a nonorthogonality graph of maximum degree two. -/
theorem exists_degree_two_maximizer (u : UnitVector E) :
  ∃ v : Configuration E (Module.finrank ℝ E + 2),
    (∀ w : Configuration E (Module.finrank ℝ E + 2), totalAngle w ≤ totalAngle v) ∧
    ∀ i, (C4NeighborCount.nonorthogonalNeighbors v i).card ≤ 2 := by
  obtain ⟨v, hmax, _, hdegree⟩ :=
    exists_sparse_maximizer (n := Module.finrank ℝ E + 2) (by omega) u
  refine ⟨v, hmax, ?_⟩
  simpa using hdegree

#print axioms exists_sparse_maximizer
#print axioms exists_degree_two_maximizer
end C4SparseConfiguration
```

1.3.16 16. C4MatrixCorank.lean

[Pinned source file](#) · SHA-256 180d00723bd73798394275b59b01dfa85a2c797b335e8267dd121993b5224cee

```
import Mathlib.LinearAlgebra.Matrix.Rank
import Mathlib.LinearAlgebra.FiniteDimensional.Lemmas
import Mathlib.LinearAlgebra.Dimension.StrongRankCondition
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Inverse
```

```

/--! Corank bounds for matrices with a nonzero interior superdiagonal. -/
namespace C4MatrixCorank

private theorem kernel_zero_of_first_two {n : ℕ} (hn : 2 ≤ n)
  (M : Matrix (Fin n) (Fin n) ℝ)
  (hband : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    (j.val + 1 < i.val ∨ i.val + 1 < j.val) → M i j = 0)
  (hsuper : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    j.val = i.val + 1 → M i j ≠ 0)
  (x : Fin n → ℝ) (hMx : M.mulVecLin x = 0)
  (h0 : x ⟨0, by omega⟩ = 0) (h1 : x ⟨1, by omega⟩ = 0) : x = 0 := by
  funext j
  have hcoord : ∀ k : ℕ, (hk : k < n) → x ⟨k, hk⟩ = 0 := by
    intro k
    induction k using Nat.strong_induction_on with
    | h k ih =>
      intro hk
      by_cases hk0 : k = 0
      · subst k
        simp using h0
      by_cases hk1 : k = 1
      · subst k
        simp using h1
      have hk2 : 2 ≤ k := by omega
      let i : Fin n := ⟨k - 1, by omega⟩
      let target : Fin n := ⟨k, hk⟩
      have hi1 : 1 ≤ i.val := by dsimp [i]; omega
      have hi2 : i.val + 1 < n := by dsimp [i]; omega
      have hrow : ∑ t : Fin n, M i t * x t = 0 := by
        have hh := congrArg (fun f : Fin n → ℝ => f i) hMx
        simp only [Matrix.mulVecLin_apply, Matrix.mulVec_apply_eq_sum,
          Pi.zero_apply] using hh
      have hsum : (∑ t : Fin n, M i t * x t) = M i target * x target := by
        classical
        apply Finset.sum_eq_single target
        · intro t _ hne
          by_cases htk : t.val < k
          · have ht0 : x t = 0 := by simp using ih t.val htk t.isLt
            simp [ht0]
          · have htail : i.val + 1 < t.val := by
              dsimp [i]
              have : t.val ≠ k := by
                intro heq
                apply hne
                exact Fin.ext heq
              omega
            simp [hband i t hi1 hi2 (Or.inr htail)]
        · simp
      have htarget : M i target * x target = 0 := by rw [← hsum]; exact hrow
      exact (mul_eq_zero.mp htarget).resolve_left
        (hsuper i target hi1 hi2 (by dsimp [i, target]; omega))
    simp using hcoord j.val j.isLt

/-- At most two independent kernel vectors can survive the interior recurrence. -/
theorem finrank_ker_le_two {n : ℕ} (hn : 2 ≤ n)
  (M : Matrix (Fin n) (Fin n) ℝ)
  (hband : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    (j.val + 1 < i.val ∨ i.val + 1 < j.val) → M i j = 0)
  (hsuper : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    j.val = i.val + 1 → M i j ≠ 0) :
  Module.finrank ℝ (LinearMap.ker M.mulVecLin) ≤ 2 := by
  let K := LinearMap.ker M.mulVecLin
  let obs : K →1[ℝ] ℝ × ℝ := {

```

```

    toFun := fun x => (x.1 ⟨0, by omega⟩, x.1 ⟨1, by omega⟩)
    map_add' := by intro x y; rfl
    map_smul' := by intro r x; rfl }
have hinj : Function.Injective obs := by
  intro x y hxy
  have h0 : (x - y).1 ⟨0, by omega⟩ = 0 := by
    have h := congrArg Prod.fst hxy
    change x.1 ⟨0, by omega⟩ = y.1 ⟨0, by omega⟩ at h
    simpa using sub_eq_zero.mpr h
  have h1 : (x - y).1 ⟨1, by omega⟩ = 0 := by
    have h := congrArg Prod.snd hxy
    change x.1 ⟨1, by omega⟩ = y.1 ⟨1, by omega⟩ at h
    simpa using sub_eq_zero.mpr h
  have hz : x - y = 0 := by
    apply Subtype.ext
    exact kernel_zero_of_first_two hn M hband hsuper (x - y).1 (x - y).2 h0 h1
  exact sub_eq_zero.mp hz
simpa using obs.firrank_le_firrank_of_injective hinj

/-- The range of a cycle-tridiagonal matrix has codimension at most two. -/
theorem firrank_range_add_two_ge {n : ℕ} (hn : 2 ≤ n)
  (M : Matrix (Fin n) (Fin n) R)
  (hband : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    (j.val + 1 < i.val ∨ i.val + 1 < j.val) → M i j = 0)
  (hsuper : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    j.val = i.val + 1 → M i j ≠ 0) :
  n ≤ Module.firrank R (LinearMap.range M.mulVecLin) + 2 := by
  have hker := firrank_ker_le_two hn M hband hsuper
  have hnull := (M.mulVecLin).firrank_range_add_firrank_ker
  have hdim : Module.firrank R (Fin n → R) = n := by simp
  omega

/-- A cycle-tridiagonal matrix with nonzero interior superdiagonal has corank at most two. -/
theorem rank_add_two_ge {n : ℕ} (hn : 2 ≤ n)
  (M : Matrix (Fin n) (Fin n) R)
  (hband : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    (j.val + 1 < i.val ∨ i.val + 1 < j.val) → M i j = 0)
  (hsuper : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    j.val = i.val + 1 → M i j ≠ 0) :
  n ≤ M.rank + 2 := by
  exact firrank_range_add_two_ge hn M hband hsuper

private theorem path_kernel_zero_of_first_one {n : ℕ} (hn : 2 ≤ n)
  (M : Matrix (Fin n) (Fin n) R)
  (hband : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    (j.val + 1 < i.val ∨ i.val + 1 < j.val) → M i j = 0)
  (hsuper : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    j.val = i.val + 1 → M i j ≠ 0)
  (hfirst : ∀ j : Fin n, 1 < j.val → M ⟨0, by omega⟩ j = 0)
  (h01 : M ⟨0, by omega⟩ ⟨1, by omega⟩ ≠ 0)
  (x : Fin n → R) (hMx : M.mulVecLin x = 0)
  (h0 : x ⟨0, by omega⟩ = 0) : x = 0 := by
  let i0 : Fin n := ⟨0, by omega⟩
  let j1 : Fin n := ⟨1, by omega⟩
  have hrow : ∑ j : Fin n, M i0 j * x j = 0 := by
    have hh := congrArg (fun f : Fin n → R => f i0) hMx
    simpa only [Matrix.mulVecLin_apply, Matrix.mulVec_apply_eq_sum,
      Pi.zero_apply] using hh
  have hsum : (∑ j : Fin n, M i0 j * x j) = M i0 j1 * x j1 := by
    classical
    apply Finset.sum_eq_single j1
    · intro j _ hne
      by_cases hj0 : j.val = 0

```

```

    · have : j = i0 := Fin.ext hj0
      subst j
      simp [i0, h0]
    · have hjgt : 1 < j.val := by
      have hj1 : j.val ≠ 1 := by
        intro heq
        apply hne
        exact Fin.ext heq
      omega
      have hz : M i0 j = 0 := by simp only [i0] using hfirst j hjgt
      simp [hz]
    · simp
have h1 : x j1 = 0 := by
  have hprod : M i0 j1 * x j1 = 0 := by rw [← hsum]; exact hrow
  exact (mul_eq_zero.mp hprod).resolve_left (by simp [i0, j1] using h01)
exact kernel_zero_of_first_two hn M hband hsuper x hMx h0 (by simp [j1] using h1)

/-- A path matrix with a nonzero first superdiagonal entry has corank at most one. -/
theorem path_finrank_ker_le_one {n : ℕ} (hn : 2 ≤ n)
  (M : Matrix (Fin n) (Fin n) ℝ)
  (hband : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    (j.val + 1 < i.val ∨ i.val + 1 < j.val) → M i j = 0)
  (hsuper : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    j.val = i.val + 1 → M i j ≠ 0)
  (hfirst : ∀ j : Fin n, 1 < j.val → M ⟨0, by omega⟩ j = 0)
  (h01 : M ⟨0, by omega⟩ ⟨1, by omega⟩ ≠ 0) :
  Module.finrank ℝ (LinearMap.ker M.mulVecLin) ≤ 1 := by
  let K := LinearMap.ker M.mulVecLin
  let obs : K →1[ℝ] ℝ := {
    toFun := fun x => x.1 ⟨0, by omega⟩
    map_add' := by intro x y; rfl
    map_smul' := by intro r x; rfl }
  have hinj : Function.Injective obs := by
    intro x y hxy
    have h0 : (x - y).1 ⟨0, by omega⟩ = 0 := by
      change x.1 ⟨0, by omega⟩ = y.1 ⟨0, by omega⟩ at hxy
      simp using sub_eq_zero.mpr hxy
    have hz : x - y = 0 := by
      apply Subtype.ext
      exact path_kernel_zero_of_first_one hn M hband hsuper hfirst h01
      (x - y).1 (x - y).2 h0
    exact sub_eq_zero.mp hz
  simp using obs.finrank_le_finrank_of_injective hinj

/-- A path matrix with the stated band and nonzero superdiagonal has rank at least `n-1`. -/
theorem path_rank_add_one_ge {n : ℕ} (hn : 2 ≤ n)
  (M : Matrix (Fin n) (Fin n) ℝ)
  (hband : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    (j.val + 1 < i.val ∨ i.val + 1 < j.val) → M i j = 0)
  (hsuper : ∀ i j : Fin n, 1 ≤ i.val → i.val + 1 < n →
    j.val = i.val + 1 → M i j ≠ 0)
  (hfirst : ∀ j : Fin n, 1 < j.val → M ⟨0, by omega⟩ j = 0)
  (h01 : M ⟨0, by omega⟩ ⟨1, by omega⟩ ≠ 0) :
  n ≤ M.rank + 1 := by
  have hker := path_finrank_ker_le_one hn M hband hsuper hfirst h01
  have hnull := (M.mulVecLin).finrank_range_add_finrank_ker
  have hdim : Module.finrank ℝ (Fin n → ℝ) = n := by simp
  change M.rank + Module.finrank ℝ (LinearMap.ker M.mulVecLin) =
    Module.finrank ℝ (Fin n → ℝ) at hnull
  omega

#print axioms finrank_ker_le_two
#print axioms finrank_range_add_two_ge

```

```
#print axioms rank_add_two_ge
#print axioms path_finrank_ker_le_one
#print axioms path_rank_add_one_ge
end C4MatrixCorank
```

1.3.17 17. C4FiveCycle.lean

[Pinned source file](#) · SHA-256 fe0bb40d9fc9b71f42d69ef814c7199db5a214bfbcee507addca3340f8d4aed7

```
import C4Pentagon

/--! The explicit coordinate five-cycle scalar inequality. -/
namespace C4FiveCycle

private theorem asin_ratio_data {x y L : ℝ} (hx0 : 0 ≤ x) (hy0 : 0 ≤ y)
  (hLpos : 0 < L) (hL2 : L ^ 2 = x ^ 2 + y ^ 2) :
  Real.sin (Real.arcsin (x / L)) = x / L ∧
  Real.cos (Real.arcsin (x / L)) = y / L ∧
  0 ≤ Real.arcsin (x / L) ∧ Real.arcsin (x / L) ≤ Real.pi / 2 := by
  have hxl : x ≤ L := by nlinarith only [hL2, hx0, hy0, hLpos, sq_nonneg y]
  have hxratio0 : 0 ≤ x / L := div_nonneg hx0 hLpos.le
  have hxratio1 : x / L ≤ 1 := (div_le_one hLpos).mpr hxl
  have hcos : Real.cos (Real.arcsin (x / L)) = y / L := by
    rw [Real.cos_arcsin]
  have hrad : 0 ≤ 1 - (x / L) ^ 2 := by nlinarith [sq_nonneg (x / L - 1)]
  have hsq : (Real.sqrt (1 - (x / L) ^ 2)) ^ 2 = (y / L) ^ 2 := by
    rw [Real.sq_sqrt hrad]
  calc
    1 - (x / L) ^ 2 = (L ^ 2 - x ^ 2) / L ^ 2 := by field_simp
    _ = (y / L) ^ 2 := by rw [div_pow, hL2]; ring
  nlinarith only [hsq, Real.sqrt_nonneg (1 - (x / L) ^ 2),
    div_nonneg hy0 hLpos.le]
  exact ⟨Real.sin_arcsin (by linarith) hxratio1, hcos,
    Real.arcsin_nonneg.mpr hxratio0, Real.arcsin_le_pi_div_two _⟩

set_option maxHeartbeats 1000000 in
/-- Five-cycle deficiency bound for the nondegenerate coordinate model. -/
theorem explicit_five_cycle {a c : ℝ} (ha0 : 0 < a) (ha1 : a < 1)
  (hc0 : 0 < c) (hc1 : c < 1) :
  Real.pi ≤
    Real.arcsin (c * Real.sqrt (1 - a ^ 2) /
      Real.sqrt (a ^ 2 + c ^ 2 - a ^ 2 * c ^ 2)) +
    Real.arcsin (a * Real.sqrt (1 - c ^ 2) /
      Real.sqrt (a ^ 2 + c ^ 2 - a ^ 2 * c ^ 2)) +
    Real.arcsin c +
    Real.arcsin (Real.sqrt (1 - a ^ 2) * Real.sqrt (1 - c ^ 2)) +
    Real.arcsin a := by
  let s := Real.sqrt (1 - a ^ 2)
  let t := Real.sqrt (1 - c ^ 2)
  let L := Real.sqrt (a ^ 2 + c ^ 2 - a ^ 2 * c ^ 2)
  let u := c * s / L
  let v := a * t / L
  let E := Real.arcsin (a * c / (1 + s * t))
  have hs0 : 0 ≤ s := Real.sqrt_nonneg _
  have ht0 : 0 ≤ t := Real.sqrt_nonneg _
  have hs2 : s ^ 2 = 1 - a ^ 2 := Real.sq_sqrt (by nlinarith)
  have ht2 : t ^ 2 = 1 - c ^ 2 := Real.sq_sqrt (by nlinarith)
  have hs1 : s ≤ 1 := by nlinarith only [hs0, hs2, sq_nonneg a]
  have ht1 : t ≤ 1 := by nlinarith only [ht0, ht2, sq_nonneg c]
  have hL2 : L ^ 2 = a ^ 2 + c ^ 2 - a ^ 2 * c ^ 2 := by
```

```

    apply Real.sq_sqrt
    nlinarith [mul_nonneg (sq_nonneg c) (by nlinarith only [ha0.le, ha1.le])]
have hLpos : 0 < L := by
  have hL0 := Real.sqrt_nonneg (a ^ 2 + c ^ 2 - a ^ 2 * c ^ 2)
  have haux := mul_nonneg (sq_nonneg c) (by nlinarith only [ha0.le, ha1.le] : 0 ≤ 1 - a ^ 2)
  dsimp [L] at *
  nlinarith only [hL0, hL2, haux, ha0]
have hxsq : L ^ 2 = (c * s) ^ 2 + a ^ 2 := by
  rw [mul_pow, hs2, hL2]
  ring
have hysq : L ^ 2 = (a * t) ^ 2 + c ^ 2 := by
  rw [mul_pow, ht2, hL2]
  ring
have hu := asin_ratio_data (mul_nonneg hc0.le hs0) ha0.le hLpos hxsq
have hv := asin_ratio_data (mul_nonneg ha0.le ht0) hc0.le hLpos hysq
have hst0 : 0 ≤ s * t := mul_nonneg hs0 ht0
have hst1 : s * t ≤ 1 := by nlinarith [mul_nonneg (sub_nonneg.mpr hs1) ht0]
have hdpos : 0 < 1 + s * t := by linarith
have hprod : (s * t) ^ 2 = (1 - a ^ 2) * (1 - c ^ 2) := by
  rw [mul_pow, hs2, ht2]
have hLrelation : L ^ 2 = 1 - (s * t) ^ 2 := by
  rw [hL2, hprod]
  ring
have hratioE0 : 0 ≤ a * c / (1 + s * t) := by positivity
have hratioE1 : a * c / (1 + s * t) ≤ 1 := by
  apply (div_le_one hdpos).mpr
  nlinarith [mul_nonneg (sub_nonneg.mpr ha1.le) hc0.le]
have hsinE : Real.sin E = a * c / (1 + s * t) :=
  Real.sin_arcsin (by linarith) hratioE1
have hE0 : 0 ≤ E := Real.arcsin_nonneg.mpr hratioE0
have hE1 : E ≤ Real.pi / 2 := Real.arcsin_le_pi_div_two _
have hsumcos : Real.cos (Real.arcsin u + Real.arcsin v) =
  a * c / (1 + s * t) := by
  rw [Real.cos_add, hu.2.1, hv.2.1, hu.1, hv.1]
  calc
    a / L * (c / L) - c * s / L * (a * t / L) =
      (a * c * (1 - s * t)) / L ^ 2 := by ring
    _ = a * c / (1 + s * t) := by
      have hLne : L ^ 2 ≠ 0 := pow_ne_zero 2 (ne_of_gt hLpos)
      field_simp
      nlinarith only [hLrelation]
have hsum0 : 0 ≤ Real.arcsin u + Real.arcsin v := add_nonneg hu.2.2.1 hv.2.2.1
have hsumpi : Real.arcsin u + Real.arcsin v ≤ Real.pi := by
  linarith [hu.2.2.2, hv.2.2.2]
have hsum : Real.arcsin u + Real.arcsin v = Real.pi / 2 - E := by
  have hcosTarget : Real.cos (Real.pi / 2 - E) = a * c / (1 + s * t) := by
    rw [Real.cos_pi_div_two_sub, hsinE]
  have htarget0 : 0 ≤ Real.pi / 2 - E := by linarith
  have htargetpi : Real.pi / 2 - E ≤ Real.pi := by linarith [Real.pi_pos]
  have h1 := Real.arccos_cos hsum0 hsumpi
  have h2 := Real.arccos_cos htarget0 htargetpi
  rw [hsumcos] at h1
  rw [hcosTarget] at h2
  linarith
have hacosS : Real.arccos s = Real.arcsin a := by
  have haasin0 : 0 ≤ Real.arcsin a := Real.arcsin_nonneg.mpr ha0.le
  have haasin1 : Real.arcsin a ≤ Real.pi := by
    linarith [Real.arcsin_le_pi_div_two a, Real.pi_pos]
  rw [← Real.arccos_cos haasin0 haasin1, Real.cos_arcsin]
have hacosT : Real.arccos t = Real.arcsin c := by
  have hcasin0 : 0 ≤ Real.arcsin c := Real.arcsin_nonneg.mpr hc0.le
  have hcasin1 : Real.arcsin c ≤ Real.pi := by
    linarith [Real.arcsin_le_pi_div_two c, Real.pi_pos]

```

```

    rw [← Real.arccos_cos hcasin0 hcasin1, Real.cos_arcsin]
    have hP := C4Pentagon.pentagon_scalar hs0 hs1 ht0 ht1
    rw [show 1 - s ^ 2 = a ^ 2 by nlinarith [hs2],
        show 1 - t ^ 2 = c ^ 2 by nlinarith [ht2],
        Real.sqrt_sq ha0.le, Real.sqrt_sq hc0.le, hacosS, hacosT] at hP
    have hst : Real.arcsin (s * t) = Real.pi / 2 - Real.arccos (s * t) :=
      Real.arcsin_eq_pi_div_two_sub_arccos _
    change Real.arccos (s * t) + E ≤ Real.arcsin a + Real.arcsin c at hP
    change Real.pi ≤ Real.arcsin u + Real.arcsin v + Real.arcsin c +
      Real.arcsin (s * t) + Real.arcsin a
    linarith only [hsum, hP, hst]

#print axioms explicit_five_cycle
end C4FiveCycle

```

1.3.18 18. C4SixCycle.lean

[Pinned source file](#) · SHA-256 2dc09cb3ac16e054a07e888b0e9aa527f541f4537c84b906b34ec0646df1be6f

```

import C4Pentagon

/--! The local six-cycle projection inequality in raw coordinates. -/
namespace C4SixCycle

private theorem arccos_ratio_data {x y L : ℝ} (hx0 : 0 ≤ x) (hy0 : 0 ≤ y)
  (hLpos : 0 < L) (hL2 : L ^ 2 = x ^ 2 + y ^ 2) :
  Real.cos (Real.arccos (x / L)) = x / L ∧
  Real.sin (Real.arccos (x / L)) = y / L ∧
  0 ≤ Real.arccos (x / L) ∧ Real.arccos (x / L) ≤ Real.pi / 2 := by
  have hXL : x ≤ L := by nlinarith only [hL2, hx0, hy0, hLpos, sq_nonneg y]
  have hxratio0 : 0 ≤ x / L := div_nonneg hx0 hLpos.le
  have hxratio1 : x / L ≤ 1 := (div_le_one hLpos).mpr hXL
  have hsin : Real.sin (Real.arccos (x / L)) = y / L := by
    rw [Real.sin_arccos]
  have hrad : 0 ≤ 1 - (x / L) ^ 2 := by nlinarith [sq_nonneg (x / L - 1)]
  have hsq : (Real.sqrt (1 - (x / L) ^ 2)) ^ 2 = (y / L) ^ 2 := by
    rw [Real.sq_sqrt hrad]
  calc
    1 - (x / L) ^ 2 = (L ^ 2 - x ^ 2) / L ^ 2 := by field_simp
    _ = (y / L) ^ 2 := by rw [div_pow, hL2]; ring
  nlinarith only [hsq, Real.sqrt_nonneg (1 - (x / L) ^ 2),
    div_nonneg hy0 hLpos.le]
  exact ⟨Real.cos_arccos (by linarith) hxratio1, hsin,
    Real.arccos_nonneg _, Real.arccos_le_pi_div_two.mpr hxratio0⟩

/-- Raw-coordinate local projection estimate for an irreducible six-cycle. -/
theorem raw_projection_inequality {a b c d u v : ℝ}
  (ha0 : 0 < a) (hb0 : 0 < b) (hc0 : 0 < c) (hd0 : 0 < d)
  (hu0 : 0 < u) (hv0 : 0 < v)
  (hleft : a ^ 2 + b ^ 2 + u ^ 2 = 1)
  (hright : c ^ 2 + d ^ 2 + v ^ 2 = 1)
  (hcross : u * v = b * c) :
  Real.arcsin (a / Real.sqrt (1 - b ^ 2)) +
  Real.arcsin (d / Real.sqrt (1 - c ^ 2)) +
  Real.arcsin (b * c /
    (Real.sqrt (1 - b ^ 2) * Real.sqrt (1 - c ^ 2))) ≤
  Real.arcsin a + Real.arcsin b + Real.arcsin c + Real.arcsin d := by
  let A := Real.sqrt (1 - b ^ 2)
  let D := Real.sqrt (1 - c ^ 2)
  let R := Real.sqrt (b ^ 2 + u ^ 2)

```

```

let B := Real.arcsin b
let C := Real.arcsin c
let U := Real.arccos (a / A)
let V := Real.arccos (d / D)
let T := Real.arccos (b / R)
have hA2 : A ^ 2 = 1 - b ^ 2 := Real.sq_sqrt (by nlinarith only [hleft, sq_nonneg a, sq_nonneg u])
have hD2 : D ^ 2 = 1 - c ^ 2 := Real.sq_sqrt (by nlinarith only [hright, sq_nonneg d, sq_nonneg v])
have hR2 : R ^ 2 = b ^ 2 + u ^ 2 := Real.sq_sqrt (by positivity)
have hA2' : A ^ 2 = a ^ 2 + u ^ 2 := by nlinarith only [hA2, hleft]
have hD2' : D ^ 2 = d ^ 2 + v ^ 2 := by nlinarith only [hD2, hright]
have hApos : 0 < A := by
  have hA0 := Real.sqrt_nonneg (1 - b ^ 2)
  nlinarith only [hA0, hA2', hA0, sq_nonneg u]
have hDpos : 0 < D := by
  have hD0 := Real.sqrt_nonneg (1 - c ^ 2)
  nlinarith only [hD0, hD2', hD0, sq_nonneg v]
have hRpos : 0 < R := by
  have hR0 := Real.sqrt_nonneg (b ^ 2 + u ^ 2)
  nlinarith only [hR0, hR2, hB0, sq_nonneg u]
have hb1 : b ≤ 1 := by nlinarith only [hleft, hb0, ha0, hu0]
have hc1 : c ≤ 1 := by nlinarith only [hright, hc0, hd0, hv0]
have hBsin : Real.sin B = b := Real.sin_arcsin (by linarith) hb1
have hCsin : Real.sin C = c := Real.sin_arcsin (by linarith) hc1
have hBcos : Real.cos B = A := Real.cos_arcsin b
have hCcos : Real.cos C = D := Real.cos_arcsin c
have hB0 : 0 ≤ B := Real.arcsin_nonneg.mpr hb0.le
have hC0 : 0 ≤ C := Real.arcsin_nonneg.mpr hc0.le
have hB1 : B ≤ Real.pi / 2 := Real.arcsin_le_pi_div_two _
have hC1 : C ≤ Real.pi / 2 := Real.arcsin_le_pi_div_two _
have hU := arccos_ratio_data ha0.le hu0.le hApos hA2'
have hV := arccos_ratio_data hd0.le hv0.le hDpos hD2'
have hT := arccos_ratio_data hb0.le hu0.le hRpos hR2
have hrel1 : Real.sin B * Real.sin T = Real.cos B * Real.sin U * Real.cos T := by
  rw [hBsin, hT.2.1, hBcos, hU.2.1, hT.1]
  field_simp
have hrel2 : Real.sin C * Real.cos T = Real.cos C * Real.sin V * Real.sin T := by
  rw [hCsin, hT.1, hCcos, hV.2.1, hT.2.1]
  field_simp
  nlinarith only [hcross]
have hproj := C4Pentagon.local_projection_inequality
  hT.2.2.1 hT.2.2.2 hB0 hB1 hC0 hc1
  hU.2.2.1 hU.2.2.2 hV.2.2.1 hV.2.2.2 hrel1 hrel2
have hsinUV : Real.sin U * Real.sin V = b * c / (A * D) := by
  rw [hU.2.1, hV.2.1, ← hcross]
  ring
have hcosBU : Real.cos B * Real.cos U = a := by
  rw [hBcos, hU.1]
  field_simp
have hcosCV : Real.cos C * Real.cos V = d := by
  rw [hCcos, hV.1]
  field_simp
rw [hsinUV, hcosBU, hcosCV] at hproj
simp only [Real.arccos_eq_pi_div_two_sub_arcsin] at hproj
change Real.arcsin (a / A) + Real.arcsin (d / D) +
  Real.arcsin (b * c / (A * D)) ≤
  Real.arcsin a + Real.arcsin b + Real.arcsin c + Real.arcsin d
dsimp [B, C] at hproj
linarith

#print axioms raw_projection_inequality
end C4SixCycle

```